



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN



LABORATORIO DE COMPUTACIÓN GRÁFICA e INTERACCIÓN HUMANO
COMPUTADORA

MANUAL TÉCNICO DE DESARROLLO.

Nombre Completo	N. Cuenta	Grupo de Laboratorio	Grupo de Teoría
Barragan Pilar Diana	318147981	03	04
Medrano Miranda Daniel Ulises	318045351	03	04
Uriarte Ortiz Enrique Yahir	318234757	02	04

EQUIPO: 04

SEMESTRE 2025-1

FECHA DE ENTREGA LÍMITE: Martes 19 de Noviembre del
2024

CALIFICACIÓN: _____

Manual técnico de desarrollo: Manual en el cual explican por medio de capturas de pantalla y bloques de código cómo se cubrieron los puntos requeridos en el apartado de “Elementos a incluir dentro del Escenario”, apartado de conclusiones individuales, conclusión grupal y Bibliografía.

Elementos dentro del escenario.

- ***El tablero está formado por una zona cuadrada de 10 recuadros de largo dando un total de 40 recuadros, cada uno creado de forma individual y tendrá 3 estados visuales diferentes.***

Para desarrollar esta actividad, nos guiamos por el planteamiento original de nuestro documento propuesta donde las casillas de cada esquina tendrían una forma cuadrada en su cara superior, mientras que las casillas intermedias serían de la mitad de ancho, como boceto de las dimensiones utilizamos geogebra para ver cómo se vería las casillas de las esquinas, para que se pudieran diferenciar de la superficie las definimos con una pequeña altura.

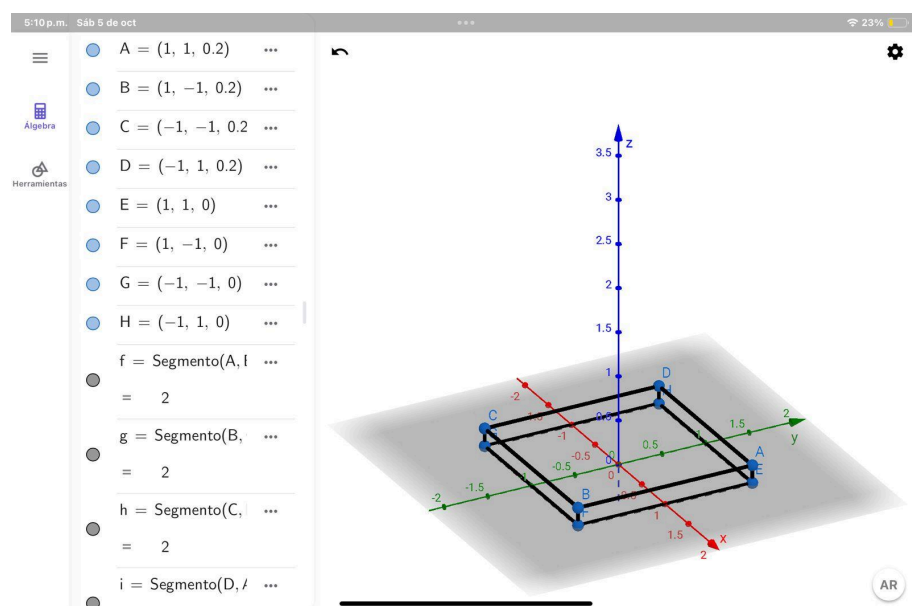


Fig.1. Boceto en Geogebra de las casillas para las esquinas.

Después pasamos a su aplicación en el código, primero definimos la implementación de las casillas esquina, estás teniendo un total de 23 vértices, cada 4 vértices definen una de las caras del prisma rectangular, también definimos desde un inicio la dirección de en la que se reflejaba la luz en estos elementos, adelantamos a la implementación de las luces para darle más realismo a la ejecución.

Para las casillas intermedias desarrollamos el mismo número de vértices, la diferencia fue que la distancia entre los vértices de las caras superior e inferior era de la mitad de distancia de las casillas de esquina. El nombre para la función de crear las casillas de esquina es ***void Casillas_Esquina()*** y para las casillas intermedias ***void Casillas_Inter()***.

Después pasamos a la definición de las 40 casillas, su posición, su escala y su rotación, para identificar mejor cada una, utilizamos los nombres alias que les asignamos en el documento propuesta, nombradas por una letra de la A a la D y un número del 1 al 9, las casillas de cada esquina siguen otro código, nombradas todas por la letra S y un número del 1 al 4.

```
void Casillas_Esquina() {
    unsigned int Casillas1_indices[] = {
        0,1,2, 0,2,3,    //Up
        4,5,6, 4,6,7,    //Down
        8,9,11, 8,10,11, //Lateral_01
        12,13,15, 12,14,15, //Lateral_02
        16,17,18, 17,18,19, //Lateral_03
        20,21,22, 21,22,23, //Lateral_04
    };

    GLfloat Casillas1_vertices[] = {
        //Up
        1.0f, 0.2f, 1.0f, 0.00f, 1.0f, 0.0f, -1.0f, 0.0f, //A - 0
        1.0f, 0.2f, -1.0f, 0.00f, 0.0f, 0.0f, -1.0f, 0.0f, //B - 1
        -1.0f, 0.2f, -1.0f, 0.79f, 0.0f, 0.0f, -1.0f, 0.0f, //C - 2
        -1.0f, 0.2f, 1.0f, 0.79f, 1.0f, 0.0f, -1.0f, 0.0f, //D - 3

        //Down
        1.0f, 0.0f, 1.0f, 0.79f, 1.0f, 0.0f, 1.0f, 0.0f, //E - 4
        1.0f, 0.0f, -1.0f, 0.79f, 0.0f, 0.0f, 1.0f, 0.0f, //F - 5
        -1.0f, 0.0f, -1.0f, 0.99f, 0.0f, 0.0f, 0.0f, 1.0f, //G - 6
        -1.0f, 0.0f, 1.0f, 0.99f, 1.0f, 0.0f, 0.0f, 1.0f, //H - 7

        //Lateral_01
        1.0f, 0.2f, 1.0f, 0.795f, 1.0f, 0.0f, -1.0f, 0.0f, //A - 8
        1.0f, 0.2f, -1.0f, 0.985f, 1.0f, 0.0f, -1.0f, 0.0f, //B - 9
        1.0f, 0.0f, 1.0f, 0.795f, 0.0f, 0.0f, -1.0f, 0.0f, //E - 10
        1.0f, 0.0f, -1.0f, 0.985f, 0.0f, 0.0f, -1.0f, 0.0f, //F - 11

        //Lateral_02
        1.0f, 0.2f, -1.0f, 0.795f, 1.0f, 0.0f, -1.0f, 0.0f, //B - 12
        -1.0f, 0.2f, -1.0f, 0.985f, 1.0f, 0.0f, -1.0f, 0.0f, //C - 13
        1.0f, 0.0f, -1.0f, 0.795f, 0.0f, 0.0f, -1.0f, 0.0f, //F - 14
        -1.0f, 0.0f, -1.0f, 0.985f, 0.0f, 0.0f, -1.0f, 0.0f, //G - 15

        //Lateral_03
        1.0f, 0.2f, 1.0f, 0.795f, 1.0f, 0.0f, -1.0f, 0.0f, //A - 16
        -1.0f, 0.2f, 1.0f, 0.985f, 1.0f, 0.0f, -1.0f, 0.0f, //D - 17
        1.0f, 0.0f, 1.0f, 0.795f, 0.0f, 0.0f, -1.0f, 0.0f, //E - 18
        -1.0f, 0.0f, 1.0f, 0.985f, 0.0f, 0.0f, -1.0f, 0.0f, //H - 19

        //Lateral_04
        -1.0f, 0.2f, -1.0f, 0.795f, 1.0f, 0.0f, -1.0f, 0.0f, //C - 20
        -1.0f, 0.2f, 1.0f, 0.985f, 1.0f, 0.0f, -1.0f, 0.0f, //D - 21
        -1.0f, 0.0f, -1.0f, 0.795f, 0.0f, 0.0f, -1.0f, 0.0f, //G - 22
        -1.0f, 0.0f, 1.0f, 0.985f, 0.0f, 0.0f, -1.0f, 0.0f, //H - 23
    };

    Mesh* Casillas1 = new Mesh();
    Casillas1->CreateMesh(Casillas1_vertices, Casillas1_indices, 192, 36);
    meshList.push_back(Casillas1);
}
```

Fig.2. Coordenadas para las casillas de cada esquina.

```

void Casillas_Inter() {
    unsigned int Casillas2_indices[] = {
        0,1,2, 0,2,3,    //Up
        4,5,6, 4,6,7,    //Down
        8,9,11, 8,10,11,  //Lateral_01
        12,13,15, 12,14,15, //Lateral_02
        16,17,18, 17,18,19, //Lateral_03
        20,21,22, 21,22,23, //Lateral_04
        8,9,11, 8,10,11,  //Lateral_01
        12,13,15, 12,14,15, //Lateral_02
        16,17,18, 17,18,19, //Lateral_03
        20,21,22, 21,22,23, //Lateral_04
    };
    GLfloat Casillas2_vertices[] = {
        //Up
        1.0f, 0.2f, 0.5f, 0.00f,0.0f, 0.0f,-1.0f,0.0f, //A - 0
        1.0f, 0.2f,-0.5f, 0.72f,0.0f, 0.0f,-1.0f,0.0f, //B - 1
        -1.0f, 0.2f,-0.5f, 0.72f,1.0f, 0.0f,-1.0f,0.0f, //C - 2
        -1.0f, 0.2f, 0.5f, 0.00f,1.0f, 0.0f,-1.0f,0.0f, //D - 3

        //Down
        1.0f, 0.0f, 0.5f, 0.75f,1.0f, 0.0f,1.0f,0.0f, //E - 4
        1.0f, 0.0f,-0.5f, 0.75f,0.0f, 0.0f,1.0f,0.0f, //F - 5
        -1.0f, 0.0f,-0.5f, 0.99f,0.0f, 0.0f,1.0f,0.0f, //G - 6
        -1.0f, 0.0f, 0.5f, 0.99f,1.0f, 0.0f,1.0f,0.0f, //H - 7

        //Lateral_01
        1.0f, 0.2f, 0.5f, 0.75f,1.0f, 0.0f,-1.0f,0.0f, //A - 8
        1.0f, 0.2f,-0.5f, 0.985f,1.0f, 0.0f,-1.0f,0.0f, //B - 9
        1.0f, 0.0f, 0.5f, 0.75f,0.0f, 0.0f,-1.0f,0.0f, //E - 10
        1.0f, 0.0f,-0.5f, 0.985f,0.0f, 0.0f,-1.0f,0.0f, //F - 11

        //Lateral_02
        1.0f, 0.2f,-0.5f, 0.75f,1.0f, 0.0f,-1.0f,0.0f, //B - 12
        -1.0f, 0.2f,-0.5f, 0.985f,1.0f, 0.0f,-1.0f,0.0f, //C - 13
        1.0f, 0.0f,-0.5f, 0.75f,0.0f, 0.0f,-1.0f,0.0f, //F - 14
        -1.0f, 0.0f,-0.5f, 0.985f,0.0f, 0.0f,-1.0f,0.0f, //G - 15

        //Lateral_03
        1.0f, 0.2f, 0.5f, 0.75f,1.0f, 0.0f,-1.0f,0.0f, //A - 16
        -1.0f, 0.2f, 0.5f, 0.985f,1.0f, 0.0f,-1.0f,0.0f, //D - 17
        1.0f, 0.0f, 0.5f, 0.75f,0.0f, 0.0f,-1.0f,0.0f, //E - 18
        -1.0f, 0.0f, 0.5f, 0.985f,0.0f, 0.0f,-1.0f,0.0f, //H - 19

        //Lateral_04
        -1.0f, 0.2f,-0.5f, 0.75f,1.0f, 0.0f,-1.0f,0.0f, //C - 20
        -1.0f, 0.2f, 0.5f, 0.985f,1.0f, 0.0f,-1.0f,0.0f, //D - 21
        -1.0f, 0.0f,-0.5f, 0.75f,0.0f, 0.0f,-1.0f,0.0f, //G - 22
        -1.0f, 0.0f, 0.5f, 0.985f,0.0f, 0.0f,-1.0f,0.0f, //H - 23
    };
    Mesh* Casillas2 = new Mesh();
    Casillas2->CreateMesh(Casillas2_vertices, Casillas2_indices, 192, 36);
    meshList.push_back(Casillas2);
}

```

Fig.3. Coordenadas para las casillas intermedias entre las de esquina.

Debido a que la captura de pantalla de código de las 40 casillas aumentaría demasiado el peso del documento, solo compartimos algunas capturas de las casillas de esquina principalmente, la estructura en todas es la misma casillas intermedias es casi la misma, ya que utilizan la misma escala definida por la primer casilla colocada, la S4, pero por cada nueva dirección para las esquinas de las casillas asignamos una rotación de 90°, las casillas que cuentan con esta rotación son la B1, C1 y D1.

```

int main() {
    mainWindow = Window(1366, 768); // 1280, 1024 or 1024, 768
    mainWindow.Initialise();

    CreateObjects();    Casillas_Esquina();    Casillas_Inter();
    Dado_4();           Dado_8();             Centro();
    CreateShaders();
}

```

Fig.4. Definición de funciones en la función main .

```

//Start - Esquina 1
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, -1.0f, 0.0f));
modellamp1 = model; modellamp2 = model; modellamp3 = model; modellamp4 = model;
model = glm::scale(model, glm::vec3(5.0f, 5.0f, 5.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
if (strcmp(casilla_Activacion, "S4") == 0 && PuedeSubir) { Casilla1_D.UseTexture(); }
else if (FASE_01) { Casilla1.UseTexture(); }
else if (!FASE_01) { Casilla1_N.UseTexture(); }
Material_opaco.UseMaterial(uniformSpecularIntensity, uniformShininess);
meshList[4]->RenderMesh();

//A1
model = glm::translate(model, glm::vec3(0.0f, 0.0f, 1.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
if (strcmp(casilla_Activacion, "A1") == 0 && PuedeSubir) { A1_D.UseTexture(); }
else if (FASE_01) { A1.UseTexture(); }
else if (!FASE_01) { A1_N.UseTexture(); }
Material_opaco.UseMaterial(uniformSpecularIntensity, uniformShininess);
meshList[5]->RenderMesh();

```

Fig.5. Definición de Casillas S4 y A1 .

```

//A9
model = glm::translate(model, glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
if (strcmp(casilla_Activacion, "A9") == 0 && PuedeSubir) { A9_D.UseTexture(); }
else if (FASE_01) { A9.UseTexture(); }
else if (!FASE_01) { A9_N.UseTexture(); }
Material_opaco.UseMaterial(uniformSpecularIntensity, uniformShininess);
meshList[5]->RenderMesh();

//Esquina 2
model = glm::translate(model, glm::vec3(0.0f, 0.0f, 1.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
if (strcmp(casilla_Activacion, "S1") == 0 && PuedeSubir) { Csl2_D.UseTexture(); }
else if (FASE_01) { Csl2.UseTexture(); }
else if (!FASE_01) { Csl2_N.UseTexture(); }
Material_opaco.UseMaterial(uniformSpecularIntensity, uniformShininess);
meshList[4]->RenderMesh();

//B1
model = glm::translate(model, glm::vec3(-1.5f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
if (strcmp(casilla_Activacion, "B1") == 0 && PuedeSubir) { B1_D.UseTexture(); }
else if (FASE_01) { B1.UseTexture(); }
else if (!FASE_01) { B1_N.UseTexture(); }
Material_opaco.UseMaterial(uniformSpecularIntensity, uniformShininess);
meshList[5]->RenderMesh();

```

Fig.6. Definición de Casillas A9, S1 y B1 .

```

//B9
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
if (strcmp(casilla_Activacion, "B9") == 0 && PuedeSubir) { B9_D.UseTexture(); }
else if (FASE_01) { B9.UseTexture(); }
else if (!FASE_01) { B9_N.UseTexture(); }
Material_opaco.UseMaterial(uniformSpecularIntensity, uniformShininess);
meshList[5]->RenderMesh();

//Esquina 3
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -1.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
if (strcmp(casilla_Activacion, "S2") == 0 && PuedeSubir) { Csl3_D.UseTexture(); }
else if (FASE_01) { Csl3.UseTexture(); }
else if (!FASE_01) { Csl3_N.UseTexture(); }
Material_opaco.UseMaterial(uniformSpecularIntensity, uniformShininess);
meshList[4]->RenderMesh();

//C1
model = glm::translate(model, glm::vec3(1.5f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
if (strcmp(casilla_Activacion, "C1") == 0 && PuedeSubir) { C1_D.UseTexture(); }
else if (FASE_01) { C1.UseTexture(); }
else if (!FASE_01) { C1_N.UseTexture(); }
Material_opaco.UseMaterial(uniformSpecularIntensity, uniformShininess);
meshList[5]->RenderMesh();

```

Fig.7. Definición de Casillas B9, S2 y C1 .

```

//C9
model = glm::translate(model, glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
if (strcmp(casilla_Activacion, "C9") == 0 && PuedeSubir) { C9_D.UseTexture(); }
else if (FASE_01) { C9.UseTexture(); }
else if (!FASE_01) { C9_N.UseTexture(); }
Material_opaco.UseMaterial(uniformSpecularIntensity, uniformShininess);
meshList[5]->RenderMesh();

//Esquina 4
model = glm::translate(model, glm::vec3(0.0f, 0.0f, 1.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
if (strcmp(casilla_Activacion, "S3") == 0 && PuedeSubir) { Casilla4_D.UseTexture(); }
else if (FASE_01) { Casilla4.UseTexture(); }
else if (!FASE_01) { Casilla4_N.UseTexture(); }
Material_opaco.UseMaterial(uniformSpecularIntensity, uniformShininess);
meshList[4]->RenderMesh();

//D1
model = glm::translate(model, glm::vec3(-1.5f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(-90.0f), glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
if (strcmp(casilla_Activacion, "D1") == 0 && PuedeSubir) { D1_D.UseTexture(); }
else if (FASE_01) { D1.UseTexture(); }
else if (!FASE_01) { D1_N.UseTexture(); }
Material_opaco.UseMaterial(uniformSpecularIntensity, uniformShininess);
meshList[5]->RenderMesh();

```

Fig.8. Definición de Casillas C9, S3 y D1 .

➤ **Aplicar 3 texturas diferentes a las casillas:**

- **Textura de base cuando estamos en ciclo de día.**
- **Textura de base cuando estamos en ciclo de noche.**
- **Textura de base cuando estamos en la casilla, independiente al ciclo.**

Para implementar este cambio de texturas primero definimos las texturas que tendría cada casilla, la estructura de todas es la siguiente, todas cuentan con el nombre del personaje de estas, una imagen del personaje y un color de fondo, las casillas que están activas durante el ciclo de día tienen el nombre de la casilla, como lo explicamos en la implementación anterior, y son de color amarillo anaranjado; las texturas iluminadas tienen agregado al nombre `_D` y son de color azul acua; y las texturas para el ciclo de noche tienen agregado al nombre `_N` y son de color morado. Además las casillas de esquina cuentan con un tono más fuerte de estos colores, esto con el objetivo de diferenciarlas un poco más. Para no aumentar el peso al documento únicamente mostramos la captura de la textura de la casilla S1. Todas las texturas fueron realizadas con ayuda del programa GIMP.

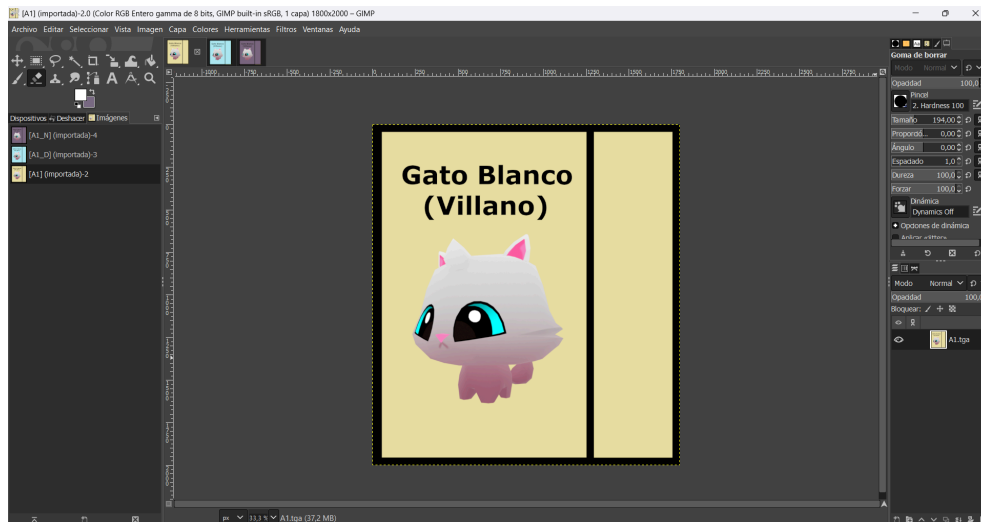


Fig.9. Textura de día casilla A1 .

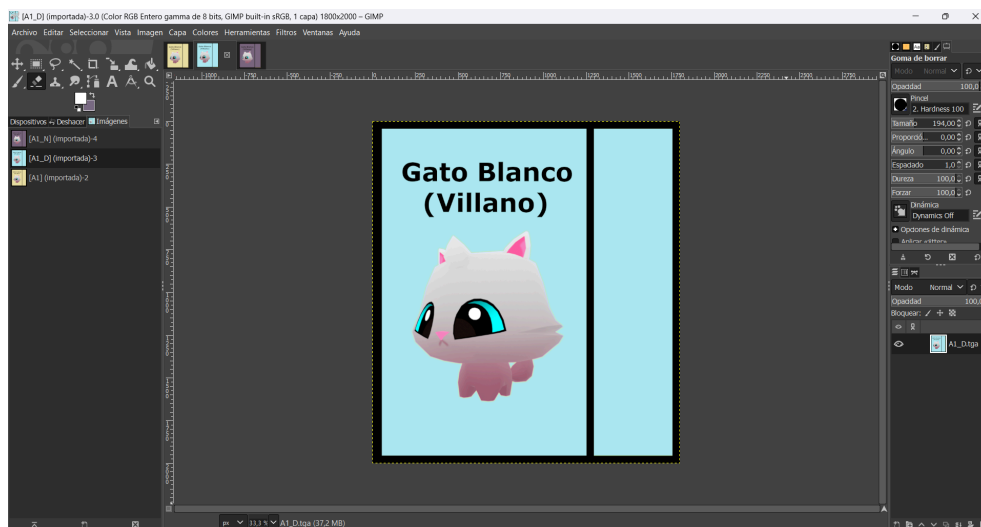


Fig.10. Textura iluminada de casilla A1 .

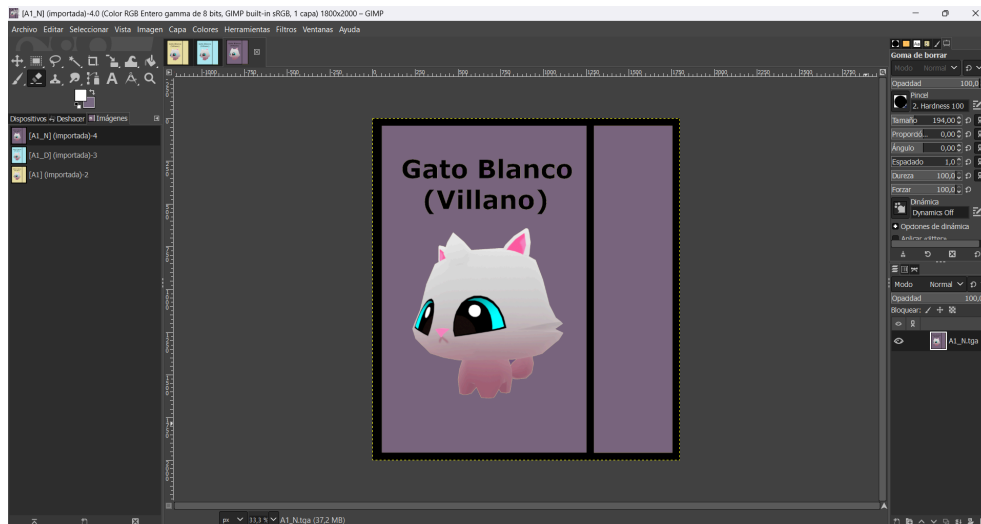


Fig.11. Textura de noche casilla A1.

Para definir las zonas en las que debía tomar la textura para cada casilla asignamos la ubicación de cada vértice en este, por ello que el diseño de las texturas de las casilla tienen un cuadrado y luego un rectángulo, ya que el rectángulo es para las demás caras, mientras que la cara superior de las figuras es la única que tendrá la imagen del personaje, la asignación de coordenadas de acuerdo a la textura se puede observar en las imágenes **Fig.2** y **Fig.3**.

Dentro del código del proyecto comenzamos a definir cada textura, así como su ubicación dentro de la carpeta de texturas.

```
//Texturas para las esquinas

//DIA
Casilla1 = Texture("Texturas/Casilla_1.tga");      Casilla1.LoadTextureA();
Csl2 = Texture("Texturas/Casilla_2.tga");          Csl2.LoadTextureA();
Csl3 = Texture("Texturas/Casilla_3.tga");          Csl3.LoadTextureA();
Casilla4 = Texture("Texturas/Casilla_4.tga");      Casilla4.LoadTextureA();
//ILUMINADOS
Casilla1_D = Texture("Texturas/Casilla_1_D.tga"); Casilla1_D.LoadTextureA();
Csl2_D = Texture("Texturas/Casilla_2_D.tga");     Csl2_D.LoadTextureA();
Csl3_D = Texture("Texturas/Casilla_3_D.tga");     Csl3_D.LoadTextureA();
Casilla4_D = Texture("Texturas/Casilla_4_D.tga"); Casilla4_D.LoadTextureA();
//NOCHE
Casilla1_N = Texture("Texturas/Casilla_1_N.tga"); Casilla1_N.LoadTextureA();
Csl2_N = Texture("Texturas/Casilla_2_N.tga");     Csl2_N.LoadTextureA();
Csl3_N = Texture("Texturas/Casilla_3_N.tga");     Csl3_N.LoadTextureA();
Casilla4_N = Texture("Texturas/Casilla_4_N.tga"); Casilla4_N.LoadTextureA();
```

Fig.12. Textura de las esquinas, de día, iluminada y de noche.

```
//Texturas para las 40 Casillas
//DIA
A1 = Texture("Texturas/A1.tga"); A1.LoadTextureA(); A2 = Texture("Texturas/A2.tga"); A2.LoadTextureA();
A3 = Texture("Texturas/A3.tga"); A3.LoadTextureA(); A4 = Texture("Texturas/A4.tga"); A4.LoadTextureA();
A5 = Texture("Texturas/A5.tga"); A5.LoadTextureA(); A6 = Texture("Texturas/A6.tga"); A6.LoadTextureA();
A7 = Texture("Texturas/A7.tga"); A7.LoadTextureA(); A8 = Texture("Texturas/A8.tga"); A8.LoadTextureA();
A9 = Texture("Texturas/A9.tga"); A9.LoadTextureA(); B1 = Texture("Texturas/B1.tga"); B1.LoadTextureA();
B2 = Texture("Texturas/B2.tga"); B2.LoadTextureA(); B3 = Texture("Texturas/B3.tga"); B3.LoadTextureA();
B4 = Texture("Texturas/B4.tga"); B4.LoadTextureA(); B5 = Texture("Texturas/B5.tga"); B5.LoadTextureA();
B6 = Texture("Texturas/B6.tga"); B6.LoadTextureA(); B7 = Texture("Texturas/B7.tga"); B7.LoadTextureA();
B8 = Texture("Texturas/B8.tga"); B8.LoadTextureA(); B9 = Texture("Texturas/B9.tga"); B9.LoadTextureA();
C1 = Texture("Texturas/C1.tga"); C1.LoadTextureA(); C2 = Texture("Texturas/C2.tga"); C2.LoadTextureA();
C3 = Texture("Texturas/C3.tga"); C3.LoadTextureA(); C4 = Texture("Texturas/C4.tga"); C4.LoadTextureA();
C5 = Texture("Texturas/C5.tga"); C5.LoadTextureA(); C6 = Texture("Texturas/C6.tga"); C6.LoadTextureA();
C7 = Texture("Texturas/C7.tga"); C7.LoadTextureA(); C8 = Texture("Texturas/C8.tga"); C8.LoadTextureA();
C9 = Texture("Texturas/C9.tga"); C9.LoadTextureA(); D1 = Texture("Texturas/D1.tga"); D1.LoadTextureA();
D2 = Texture("Texturas/D2.tga"); D2.LoadTextureA(); D3 = Texture("Texturas/D3.tga"); D3.LoadTextureA();
D4 = Texture("Texturas/D4.tga"); D4.LoadTextureA(); D5 = Texture("Texturas/D5.tga"); D5.LoadTextureA();
D6 = Texture("Texturas/D6.tga"); D6.LoadTextureA(); D7 = Texture("Texturas/D7.tga"); D7.LoadTextureA();
D8 = Texture("Texturas/D8.tga"); D8.LoadTextureA(); D9 = Texture("Texturas/D9.tga"); D9.LoadTextureA();
```

Fig.13. Textura de todas las casillas intermedias, de día.

```
//ILUMINADOS
A1_D = Texture("Textures/A1_D.tga"); A1_D.LoadTextureA(); A2_D = Texture("Textures/A2_D.tga"); A2_D.LoadTextureA();
A3_D = Texture("Textures/A3_D.tga"); A3_D.LoadTextureA(); A4_D = Texture("Textures/A4_D.tga"); A4_D.LoadTextureA();
A5_D = Texture("Textures/A5_D.tga"); A5_D.LoadTextureA(); A6_D = Texture("Textures/A6_D.tga"); A6_D.LoadTextureA();
A7_D = Texture("Textures/A7_D.tga"); A7_D.LoadTextureA(); A8_D = Texture("Textures/A8_D.tga"); A8_D.LoadTextureA();
A9_D = Texture("Textures/A9_D.tga"); A9_D.LoadTextureA(); B1_D = Texture("Textures/B1_D.tga"); B1_D.LoadTextureA();
B2_D = Texture("Textures/B2_D.tga"); B2_D.LoadTextureA(); B3_D = Texture("Textures/B3_D.tga"); B3_D.LoadTextureA();
B4_D = Texture("Textures/B4_D.tga"); B4_D.LoadTextureA(); B5_D = Texture("Textures/B5_D.tga"); B5_D.LoadTextureA();
B6_D = Texture("Textures/B6_D.tga"); B6_D.LoadTextureA(); B7_D = Texture("Textures/B7_D.tga"); B7_D.LoadTextureA();
B8_D = Texture("Textures/B8_D.tga"); B8_D.LoadTextureA(); B9_D = Texture("Textures/B9_D.tga"); B9_D.LoadTextureA();
C1_D = Texture("Textures/C1_D.tga"); C1_D.LoadTextureA(); C2_D = Texture("Textures/C2_D.tga"); C2_D.LoadTextureA();
C3_D = Texture("Textures/C3_D.tga"); C3_D.LoadTextureA(); C4_D = Texture("Textures/C4_D.tga"); C4_D.LoadTextureA();
C5_D = Texture("Textures/C5_D.tga"); C5_D.LoadTextureA(); C6_D = Texture("Textures/C6_D.tga"); C6_D.LoadTextureA();
C7_D = Texture("Textures/C7_D.tga"); C7_D.LoadTextureA(); C8_D = Texture("Textures/C8_D.tga"); C8_D.LoadTextureA();
C9_D = Texture("Textures/C9_D.tga"); C9_D.LoadTextureA(); D1_D = Texture("Textures/D1_D.tga"); D1_D.LoadTextureA();
D2_D = Texture("Textures/D2_D.tga"); D2_D.LoadTextureA(); D3_D = Texture("Textures/D3_D.tga"); D3_D.LoadTextureA();
D4_D = Texture("Textures/D4_D.tga"); D4_D.LoadTextureA(); D5_D = Texture("Textures/D5_D.tga"); D5_D.LoadTextureA();
D6_D = Texture("Textures/D6_D.tga"); D6_D.LoadTextureA(); D7_D = Texture("Textures/D7_D.tga"); D7_D.LoadTextureA();
D8_D = Texture("Textures/D8_D.tga"); D8_D.LoadTextureA(); D9_D = Texture("Textures/D9_D.tga"); D9_D.LoadTextureA();
```

Fig.14. Textura de todas las casillas intermedias, iluminadas.

```
//NOCHE
A1_N = Texture("Textures/A1_N.tga"); A1_N.LoadTextureA(); A2_N = Texture("Textures/A2_N.tga"); A2_N.LoadTextureA();
A3_N = Texture("Textures/A3_N.tga"); A3_N.LoadTextureA(); A4_N = Texture("Textures/A4_N.tga"); A4_N.LoadTextureA();
A5_N = Texture("Textures/A5_N.tga"); A5_N.LoadTextureA(); A6_N = Texture("Textures/A6_N.tga"); A6_N.LoadTextureA();
A7_N = Texture("Textures/A7_N.tga"); A7_N.LoadTextureA(); A8_N = Texture("Textures/A8_N.tga"); A8_N.LoadTextureA();
A9_N = Texture("Textures/A9_N.tga"); A9_N.LoadTextureA(); B1_N = Texture("Textures/B1_N.tga"); B1_N.LoadTextureA();
B2_N = Texture("Textures/B2_N.tga"); B2_N.LoadTextureA(); B3_N = Texture("Textures/B3_N.tga"); B3_N.LoadTextureA();
B4_N = Texture("Textures/B4_N.tga"); B4_N.LoadTextureA(); B5_N = Texture("Textures/B5_N.tga"); B5_N.LoadTextureA();
B6_N = Texture("Textures/B6_N.tga"); B6_N.LoadTextureA(); B7_N = Texture("Textures/B7_N.tga"); B7_N.LoadTextureA();
B8_N = Texture("Textures/B8_N.tga"); B8_N.LoadTextureA(); B9_N = Texture("Textures/B9_N.tga"); B9_N.LoadTextureA();
C1_N = Texture("Textures/C1_N.tga"); C1_N.LoadTextureA(); C2_N = Texture("Textures/C2_N.tga"); C2_N.LoadTextureA();
C3_N = Texture("Textures/C3_N.tga"); C3_N.LoadTextureA(); C4_N = Texture("Textures/C4_N.tga"); C4_N.LoadTextureA();
C5_N = Texture("Textures/C5_N.tga"); C5_N.LoadTextureA(); C6_N = Texture("Textures/C6_N.tga"); C6_N.LoadTextureA();
C7_N = Texture("Textures/C7_N.tga"); C7_N.LoadTextureA(); C8_N = Texture("Textures/C8_N.tga"); C8_N.LoadTextureA();
C9_N = Texture("Textures/C9_N.tga"); C9_N.LoadTextureA(); D1_N = Texture("Textures/D1_N.tga"); D1_N.LoadTextureA();
D2_N = Texture("Textures/D2_N.tga"); D2_N.LoadTextureA(); D3_N = Texture("Textures/D3_N.tga"); D3_N.LoadTextureA();
D4_N = Texture("Textures/D4_N.tga"); D4_N.LoadTextureA(); D5_N = Texture("Textures/D5_N.tga"); D5_N.LoadTextureA();
D6_N = Texture("Textures/D6_N.tga"); D6_N.LoadTextureA(); D7_N = Texture("Textures/D7_N.tga"); D7_N.LoadTextureA();
D8_N = Texture("Textures/D8_N.tga"); D8_N.LoadTextureA(); D9_N = Texture("Textures/D9_N.tga"); D9_N.LoadTextureA();
```

Fig.15. Textura de todas las casillas intermedias, de noche.

Para controlar el cambio de texturas definimos una variable booleana llamada **FASE_01**, la intención de esta variable es cambiar a **true** cuando es de día y si es de noche ser **false**, para el cambio de textura a iluminada depende de 2 variables que más adelante se definieron, la primera es un código de tipo carácter, su valor cambia al del código de la casilla en que se encuentra llamada **casilla_Activación** y la segunda de la que depende es una que indica cuando el modelo del personaje de la casilla puede subir, esto nos asegura que el cambio de textura ha iluminado solo se realizará hasta que el personaje llegue a la casilla, más adelante se explicara detalladamente como obtiene estos valores dichas variables. En las imágenes **Fig.5**, **Fig.6**, **Fig.7**, **Fig.8**, se observa esta implementación antes de la asignación de material opaco y el modelo implementado.

```
// --- Iluminacion ---
float T_Transcurrido; bool FASE_01;
const float TIEMPO_CAMBIO = 1000.0f;
```

Fig.16. Definición de la variable para realizar el cambio de texturas .

- **Interacción especial.** Al terminar el avatar el recorrido en la casilla se mostrará a un lado de la casilla un modelo 3d del elemento que se está mostrando en la textura base de la casilla, el modelo aparecerá desde abajo del tablero hacia arriba subiendo y girando, se mostrará por 3 segundos y se volverá a “esconder” bajando.

Para la implementación de esta funcionalidad definimos las variable para estos

movimientos, primero una variable de rotación para cada modelo, puesto que con cada una de ellas es que más adelante lograremos que el modelo suba realizando un giro.

```
// Variables para la Interacción de los modelos por movimiento.
float ultimaActivacion; bool avanza2;
float rotA1 = 0.0f, rotA2 = 0.0f, rotA3 = 0.0f, rotA4 = 0.0f, rotA5 = 0.0f, rotA6 = 0.0f, rotA7 = 0.0f, rotA8 = 0.0f, rotA9 = 0.0f, rotS1 = 0.0f;
float rotB1 = 0.0f, rotB2 = 0.0f, rotB3 = 0.0f, rotB4 = 0.0f, rotB5 = 0.0f, rotB6 = 0.0f, rotB7 = 0.0f, rotB8 = 0.0f, rotB9 = 0.0f, rotS2 = 0.0f;
float rotC1 = 0.0f, rotC2 = 0.0f, rotC3 = 0.0f, rotC4 = 0.0f, rotC5 = 0.0f, rotC6 = 0.0f, rotC7 = 0.0f, rotC8 = 0.0f, rotC9 = 0.0f, rotS3 = 0.0f;
float rotD1 = 0.0f, rotD2 = 0.0f, rotD3 = 0.0f, rotD4 = 0.0f, rotD5 = 0.0f, rotD6 = 0.0f, rotD7 = 0.0f, rotD8 = 0.0f, rotD9 = 0.0f;
```

Fig.17. Definición de las variables de rotación de cada modelo.

Posteriormente definimos las variables a utilizar para lograr el movimiento de los modelos, primero movY[nombre_casilla] que son todas las variables que vamos a ocupar para subir el modelo moviendolo en el eje Y, luego las variables de tipo arreglo de caracteres para definir que la última casilla que se implementó sea la que queremos, también se agregan variables de bandera para saber cuando avanza cada modelo y hasta qué punto se detiene, por último guardamos cuando sucedió la última activación de la casilla.

Realizamos la misma definición de variables para cada modelo de su respectivo universo.

```
// --- Casillas de la CHICAS SUPERPODEROSAS ---
float movYA1, movYA2, movYA3, movYA4, movYA5, movYA6, movYA7, movYA8, movYA9, movYS1, movYB1, movYB2, movYB3;
static char ultimaCasillaGato[3] = ""; bool avanzaGato = true; float ultimaActivacionGato = 0.0f;
static char ultimaCasillaOcti[3] = ""; bool avanzaOcti = true; float ultimaActivacionOcti = 0.0f;
static char ultimaCasillaArbol[3] = ""; bool avanzaArbol = true; float ultimaActivacionArbol = 0.0f;
static char ultimaCasillaBurbuja[3] = ""; bool avanzaBurbuja = true; float ultimaActivacionBurbuja = 0.0f;
static char ultimaCasillaBellota[3] = ""; bool avanzaBellota = true; float ultimaActivacionBellota = 0.0f;
static char ultimaCasillaMojoJojo[3] = ""; bool avanzaMojoJojo = true; float ultimaActivacionMojoJojo = 0.0f;
static char ultimaCasillaPeludito[3] = ""; bool avanzaPeludito = true; float ultimaActivacionPeludito = 0.0f;
static char ultimaCasillaPrincesa[3] = ""; bool avanzaPrincesa = true; float ultimaActivacionPrincesa = 0.0f;
static char ultimaCasillaProfesor[3] = ""; bool avanzaProfesor = true; float ultimaActivacionProfesor = 0.0f;
static char ultimaCasillaSedusa[3] = ""; bool avanzaSedusa = true; float ultimaActivacionSedusa = 0.0f;
static char ultimaCasillaCasaCSP[3] = ""; bool avanzaCasaCSP = true; float ultimaActivacionCasaCSP = 0.0f;
static char ultimaCasillaEdificioCSP[3] = ""; bool avanzaEdificioCSP = true; float ultimaActivacionEdificioCSP = 0.0f;
static char ultimaCasillaAlcaldia[3] = ""; bool avanzaAlcaldia = true; float ultimaActivacionAlcaldia = 0.0f;
```

Fig.18. Definición de las variables del movimiento en Y y .

```
// --- Casillas de BIG HERO 6 ---
float movYB4, movYB5, movYB6, movYB7, movYB8, movYB9, movYS2, movYC1, movYC2, movYC3, movYC4, movYC5, movYC6;
static char ultimaCasillaHiroHamada_M[3] = ""; bool avanzaHiroHamada_M = true; float ultimaActivacionHiroHamada_M = 0.0f;
static char ultimaCasillaPlanta1_M[3] = ""; bool avanzaPlanta1_M = true; float ultimaActivacionPlanta1_M = 0.0f;
static char ultimaCasillaAutoChico_M[3] = ""; bool avanzaAutoChico_M = true; float ultimaActivacionAutoChico_M = 0.0f;
static char ultimaCasillaGogoTomago_M[3] = ""; bool avanzaGogoTomago_M = true; float ultimaActivacionGogoTomago_M = 0.0f;
static char ultimaCasillaMotocicleta_M[3] = ""; bool avanzaMotocicleta_M = true; float ultimaActivacionMotocicleta_M = 0.0f;
static char ultimaCasillaGoldenGate_M[3] = ""; bool avanzaGoldenGate_M = true; float ultimaActivacionGoldenGate_M = 0.0f;
static char ultimaCasillaPlanta2_M[3] = ""; bool avanzaPlanta2_M = true; float ultimaActivacionPlanta2_M = 0.0f;
static char ultimaCasillaBaymaxSuperArmadura_M[3] = ""; bool avanzaBaymaxSuperArmadura_M = true; float ultimaActivacionBaymaxSuperArmadura_M = 0.0f;
static char ultimaCasillaAutoGrande_M[3] = ""; bool avanzaAutoGrande_M = true; float ultimaActivacionAutoGrande_M = 0.0f;
static char ultimaCasillaFred_M[3] = ""; bool avanzaFred_M = true; float ultimaActivacionFred_M = 0.0f;
static char ultimaCasillaPlanta3_M[3] = ""; bool avanzaPlanta3_M = true; float ultimaActivacionPlanta3_M = 0.0f;
static char ultimaCasillaCasaHiro_M[3] = ""; bool avanzaCasaHiro_M = true; float ultimaActivacionCasaHiro_M = 0.0f;
static char ultimaCasillaMegabot_M[3] = ""; bool avanzaMegabot_M = true; float ultimaActivacionMegabot_M = 0.0f;
```

Fig.19. Definición de las variables del movimiento en Y y .

```
// --- Casillas de PHINEAS Y FERB ---
float movYC7, movYC8, movYC9, movYS3, movYD1, movYD2, movYD3, movYD4, movYD5, movYD6, movYD7, movYD8, movYD9;
static char ultimaCasillaPhineas[3] = ""; bool avanzaPhineas = true; float ultimaActivacionPhineas = 0.0f;
static char ultimaCasillaOrnitocyborg[3] = ""; bool avanzaOrnitocyborg = true; float ultimaActivacionOrnitocyborg = 0.0f;
static char ultimaCasillaIsabella[3] = ""; bool avanzaIsabella = true; float ultimaActivacionIsabella = 0.0f;
static char ultimaCasillaEDoofenshmirtz[3] = ""; bool avanzaEDoofenshmirtz = true; float ultimaActivacionEDoofenshmirtz = 0.0f;
static char ultimaCasillaPeterPanda[3] = ""; bool avanzaPeterPanda = true; float ultimaActivacionPeterPanda = 0.0f;
static char ultimaCasillaDoofenshmirtz[3] = ""; bool avanzaDoofenshmirtz = true; float ultimaActivacionDoofenshmirtz = 0.0f;
static char ultimaCasillaTerryTortuga[3] = ""; bool avanzaTerryTortuga = true; float ultimaActivacionTerryTortuga = 0.0f;
static char ultimaCasillaCandace[3] = ""; bool avanzaCandace = true; float ultimaActivacionCandace = 0.0f;
static char ultimaCasillaDoofenshmirtz2D[3] = ""; bool avanzaDoofenshmirtz2D = true; float ultimaActivacionDoofenshmirtz2D = 0.0f;
static char ultimaCasillaBuford[3] = ""; bool avanzaBuford = true; float ultimaActivacionBuford = 0.0f;
static char ultimaCasillaPinkyChihuahua[3] = ""; bool avanzaPinkyChihuahua = true; float ultimaActivacionPinkyChihuahua = 0.0f;
static char ultimaCasillaCarl[3] = ""; bool avanzaCarl = true; float ultimaActivacionCarl = 0.0f;
static char ultimaCasillaFerb[3] = ""; bool avanzaFerb = true; float ultimaActivacionFerb = 0.0f;
```

Fig.20. Definición de las variables del movimiento en Y y .

```
// Modelos alrededor del tablero
Model Gato, Octi, Arbol, Burbuja, Bellota, Mojo, Peludito, Princesa, Profesor, Sedusa, Casa, Edificio, Alcaldia, Phineas, Ornitocyborg,
Isabella, PeterPanda, Doofenshmirtz, TerryTortuga, Candace, Doofenshmirtz2D, Buford, PinkyChihuahua, Carl, Ferb,
HiroHamada_M, Planta1_M, AutoChico_M, GogoTomago_M, Motocicleta_M, GoldenGate_M, Planta2_M, BaymaxSuperArmadura_M, AutoGrande_M,
Fred_M, Planta3_M, CasaHiro_M, Megabot_M, Lampara, Lampara2, esf;
```

Fig.21. Definición de los modelos que estarán alrededor del tablero .

```
Gato = Model(); Gato.LoadModel("Models/gato.obj");
Octi = Model(); Octi.LoadModel("Models/octi.obj");
Arbol = Model(); Arbol.LoadModel("Models/arbol.obj");
Burbuja = Model(); Burbuja.LoadModel("Models/burbuja.obj");
Bellota = Model(); Bellota.LoadModel("Models/bellota.obj");
Mojo = Model(); Mojo.LoadModel("Models/mojo.obj");
Peludito = Model(); Peludito.LoadModel("Models/peludito.obj");
Princesa = Model(); Princesa.LoadModel("Models/princesa.obj");
Profesor = Model(); Profesor.LoadModel("Models/profesor.obj");
Sedusa = Model(); Sedusa.LoadModel("Models/sedusa.obj");
Casa = Model(); Casa.LoadModel("Models/casa.obj");
Edificio = Model(); Edificio.LoadModel("Models/edificio.obj");
Alcaldia = Model(); Alcaldia.LoadModel("Models/alcaldia.obj");
HiroHamada_M = Model(); HiroHamada_M.LoadModel("Models/HiroHamada/hiro.obj");
Planta1_M = Model(); Planta1_M.LoadModel("Models/Planta1/Phormium_1.obj");
AutoChico_M = Model(); AutoChico_M.LoadModel("Models/AutoChico/isetta.obj");
GogoTomago_M = Model(); GogoTomago_M.LoadModel("Models/GogoTomago/gogo.obj");
Motocicleta_M = Model(); Motocicleta_M.LoadModel("Models/Motocicleta/motocicleta.obj");
GoldenGate_M = Model(); GoldenGate_M.LoadModel("Models/GoldenGate/golden gate bridge.obj");
Planta2_M = Model(); Planta2_M.LoadModel("Models/Planta2/hoewa_Forsteriana_1.obj");
BaymaxSuperArmadura_M = Model(); BaymaxSuperArmadura_M.LoadModel("Models/BaymaxSuperArmadura/baymax_super.obj");
AutoGrande_M = Model(); AutoGrande_M.LoadModel("Models/AutoGrande/500_fiat_lod_a.obj");
Fred_M = Model(); Fred_M.LoadModel("Models/Fred/fred.obj");
Planta3_M = Model(); Planta3_M.LoadModel("Models/Planta3/Planta3.obj");
CasaHiro_M = Model(); CasaHiro_M.LoadModel("Models/CasaHiro/CasaHiro.obj");
Megabot_M = Model(); Megabot_M.LoadModel("Models/Megabot/MegaBot.obj");
Phineas = Model(); Phineas.LoadModel("Models/PHINEAS.obj");
Ornitocyborg = Model(); Ornitocyborg.LoadModel("Models/Ornitocyborg.obj");
Isabella = Model(); Isabella.LoadModel("Models/Isabella.obj");
PeterPanda = Model(); PeterPanda.LoadModel("Models/PeterPanda.obj");
Doofenshmirtz = Model(); Doofenshmirtz.LoadModel("Models/Doofenshmirtz.obj");
TerryTortuga = Model(); TerryTortuga.LoadModel("Models/TerryTortuga.obj");
Candace = Model(); Candace.LoadModel("Models/Candace.obj");
Doofenshmirtz2D = Model(); Doofenshmirtz2D.LoadModel("Models/Doofenshmirtz2D.obj");
Buford = Model(); Buford.LoadModel("Models/Buford.obj");
PinkyChihuahua = Model(); PinkyChihuahua.LoadModel("Models/PinkyChihuahua.obj");
Carl = Model(); Carl.LoadModel("Models/Carl.obj");
Ferb = Model(); Ferb.LoadModel("Models/Ferb.obj");
Lampara = Model(); Lampara.LoadModel("Models/fobj_lamp_MODFIC.obj");
Lampara2 = Model(); Lampara2.LoadModel("Models/obj_0401_street_lamp.obj");
```

Fig.21. Importamos y especificamos la ruta de los modelos alrededor del tablero .

```
// Variables para la Interacción de los modelos por movimiento.
ultimaActivacion = 0.0f; avanza2 = true;
movYA1 = 0.0f; movYA2 = 0.0f; movYA3 = 0.0f; movYA4 = 0.0f; movYA5 = 0.0f; movYA6 = 0.0f; movYA7 = 0.0f; movYA8 = 0.0f; movYA9 = 0.0f; movYS1 = 0.0f;
movYB1 = 0.0f; movYB2 = 0.0f; movYB3 = 0.0f; movYB4 = 0.0f; movYB5 = 0.0f; movYB6 = 0.0f; movYB7 = 0.0f; movYB8 = 0.0f; movYB9 = 0.0f; movYS2 = 0.0f;
movYC1 = 0.0f; movYC2 = 0.0f; movYC3 = 0.0f; movYC4 = 0.0f; movYC5 = 0.0f; movYC6 = 0.0f; movYC7 = 0.0f; movYC8 = 0.0f; movYC9 = 0.0f; movYS3 = 0.0f;
movYD1 = 0.0f; movYD2 = 0.0f; movYD3 = 0.0f; movYD4 = 0.0f; movYD5 = 0.0f; movYD6 = 0.0f; movYD7 = 0.0f; movYD8 = 0.0f; movYD9 = 0.0f;
```

Fig.22. Inicialización de las variables a utilizar en cero .

Luego de haber declarado todas las variables momentos antes en el código ya comenzamos a implementar la interacción de que los modelos suban dando un giro, esto lo realizamos con la ayuda de la sentencia if. En el primero de los ifs como se puede ver en la **Fig. 23** comparamos si la casilla que se debe activar es A1, si no es el caso entonces pasamos a la siguiente casilla, claro que esto después de haber verificado que no sé de el caso en que se presiona la tecla para tirar los dados antes de culminar los 3 segundos que tarda en subir nuestro modelo, si es que esto ocurre entonces guardamos el modelo de nuevo y le asignamos nuevos valores a las variables que nos indiquen que se realizará una tirada nuevamente.

Después en el siguiente if simplemente le indicamos a la variable ultimaCasillaGato que es la A1, después verificamos si se cumplen las condiciones para que el modelo suba, las cuales son que la variable PuedeSubir tenga el valor de true al igual que la bandera de avanzaGato, esto también lo implementamos para la espera de tres segundos en que el modelo debe de subir y finalmente solo se sube el modelo correspondiente con un incremento de movimiento en la coordenada Y.

```
// --- Casillas de la CHICAS SUPERPODEROSAS ---

//GATO
if (strcmp(casilla_Activacion, "A1") == 0) {
    strcpy_s(ultimaCasillaGato, sizeof(ultimaCasillaGato), "A1");
    if (PuedeSubir == true && avanzaGato) { // Subiendo
        if (movYA1 < 6.25f) { movYA1 += 0.3 * movOffsetHeli * deltaTime; rotA1 += 17.0 * deltaTime; }
        else { avanzaGato = false; ultimaActivacionGato = now; }
    }
    else if (!avanzaGato && (now - ultimaActivacionGato) >= 3.0f) { // Espera de 3 segundos
        if (movYA1 > 0.0f) { movYA1 -= 0.3 * movOffsetHeli * deltaTime; rotA1 -= 17.0 * deltaTime; }
    }
}
else if (strcmp(ultimaCasillaGato, "A1") == 0) { // Tirada de dados antes de los 3 segundos
    if (movYA1 > 0.0f) { movYA1 -= 0.3 * movOffsetHeli * deltaTime; rotA1 -= 17.0 * deltaTime; }
    else { strcpy_s(ultimaCasillaGato, sizeof(ultimaCasillaGato), ""); avanzaGato = true; }
}
```

Fig.23. Interacción de subir el modelo de la casilla correspondiente al mundo de las chicas superpoderosas.

```
// --- Casillas de BIG HERO 6 ---

//Hiro Hamada
if (strcmp(casilla_Activacion, "B4") == 0) {
    strcpy_s(ultimaCasillaHiroHamada_M, sizeof(ultimaCasillaHiroHamada_M), "B4");
    if (PuedeSubir == true && avanzaHiroHamada_M) {
        if (movYB4 < 12.25f) { movYB4 += 0.3 * movOffsetHeli * deltaTime; rotB4 += 13.2 * deltaTime; }
        else { avanzaHiroHamada_M = false; ultimaActivacionHiroHamada_M = now; }
    }
    else if (!avanzaHiroHamada_M && (now - ultimaActivacionHiroHamada_M) >= 3.0f) { // Espera de 3 segundos
        if (movYB4 > 0.0f) { movYB4 -= 0.3 * movOffsetHeli * deltaTime; rotB4 -= 13.2 * deltaTime; }
    }
}
else if (strcmp(ultimaCasillaHiroHamada_M, "B4") == 0) {
    if (movYB4 > 0.0f) { movYB4 -= 0.3 * movOffsetHeli * deltaTime; rotB4 -= 13.2 * deltaTime; }
    else { strcpy_s(ultimaCasillaHiroHamada_M, sizeof(ultimaCasillaHiroHamada_M), ""); avanzaHiroHamada_M = true; }
}
```

Fig.24. Interacción de subir el modelo de la casilla correspondiente al mundo de Big Hero 6.

```
// --- Casillas de PHINEAS Y FERB ---

//PHINEAS
if (strcmp(casilla_Activacion, "C7") == 0) {
    strcpy_s(ultimaCasillaPhineas, sizeof(ultimaCasillaPhineas), "C7");
    if (PuedeSubir == true && avanzaPhineas) {
        if (movYC7 < 6.55f) { movYC7 += 0.3 * movOffsetHeli * deltaTime; rotC7 += 16.6 * deltaTime;}
        else {avanzaPhineas = false; ultimaActivacionPhineas = now;}}
    else if (!avanzaPhineas && (now - ultimaActivacionPhineas) >= 3.0f) { // Espera de 3 segundos
        if (movYC7 > 0.0f) { movYC7 -= 0.3 * movOffsetHeli * deltaTime; rotC7 -= 16.6 * deltaTime; }
    }
}

else if (strcmp(ultimaCasillaPhineas, "C7") == 0) {
    if (movYC7 > 0.0f) { movYC7 -= 0.3 * movOffsetHeli * deltaTime; rotC7 -= 16.6 * deltaTime;}
    else {strcpy_s(ultimaCasillaPhineas, sizeof(ultimaCasillaPhineas), "");avanzaPhineas = true;}
}
}
```

Fig.25. Interacción de subir el modelo de la casilla correspondiente al mundo de Phineas y Ferb.

En esta parte el código simplemente realizamos las adecuaciones pertinentes de cada uno de los modelos para que se encontraran en la posición y escala que se requería a un lado de la casilla pertinente con su respectiva textura, haciendo uso como en las clases de transformaciones como el translate para posicionar el modelo de forma adecuada, rotate para que se encuentre de frente a la dirección correcta y scale para que todo los elementos tengan un tamaño convencional y concordante entre ellos.

```
// --- MODELOS CHICAS SUPER PODEROSAS ---

model = modelaux; // Gato
model = glm::translate(model, glm::vec3(8.0f, movYA1 - 7.0f, 7.0f));
model = glm::rotate(model, -90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, rotA1 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.3f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Gato.RenderModel();

model = modelaux; // Octi
model = glm::translate(model, glm::vec3(8.0f, movYA2 - 7.0f, 12.5f));
model = glm::rotate(model, -90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, rotA2 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Octi.RenderModel();
```

Fig.26. Ajuste de los modelos correspondientes al mundo de Las Chicas Superpoderosas.

```
// --- MODELOS BIG HERO 6 ---

model = glm::mat4(1.0); // Hiro Hamada
model = glm::translate(model, glm::vec3(-23.0f, movYB4 - 13.2f, 65.0f));
model = glm::scale(model, glm::vec3(1.5f, 1.5f, 1.5f));
model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, rotB4 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
HiroHamada_M.RenderModel();

model = glm::mat4(1.0); // Planta1
model = glm::translate(model, glm::vec3(-28.0f, movYB5 - 13.2f, 65.0f));
model = glm::scale(model, glm::vec3(0.6f, 0.6f, 0.6f));
model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, rotB5 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Planta1_M.RenderModel();
```

Fig.27. Ajuste de los modelos correspondientes al mundo de Big Hero 6.

```
// --- MODELOS PHINEAS Y FERB -----

model = modelaux; // Phineas
model = glm::translate(model, glm::vec3(-65.0f, movYC7 - 7.5f, 17.5f));
model = glm::rotate(model, 90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, rotC7 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.25f, 0.25f, 0.25f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Phineas.RenderModel();

model = modelaux; // Ornitocborg
model = glm::translate(model, glm::vec3(-65.0f, movYC8 - 7.5f, 12.5f));
model = glm::rotate(model, 90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, rotC8 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.125f, 0.125f, 0.125f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Ornitocborg.RenderModel();
```

Fig.28. Ajuste de los modelos correspondientes al mundo de Phineas y Ferb.

Recorrido en el tablero:

- Se implementará el lanzamiento de dos dados de forma simultánea, un dado de 4 caras y un dado de 8 caras. Los dados deben de empezar flotando al centro del tablero, caer y al entrar en contacto con el tablero quedar con un número siempre en la cara superior (simulando la física real de la tirada de estos dos dados).

Para implementar el movimiento en el tablero a partir de tirar dos dados uno de ocho caras y otro de 4 caras primero declaramos las variables que nos van a ayudar a realizar las respectivas rotaciones del dado para obtener cada una de las caras correspondientes.

```
// Variables para rotacion de los Dados.
float movDado, movOffsetDado;
float RotacionX4, RotacionZ4, RotacionX8, RotacionZ8;
bool DadoRotando, DadoPiso;
```

Fig.29. Declaración de variables para la rotación de los dados.

Más adelante creamos los dados con ayuda de triángulos creando una función especialmente para esto, entonces lo primero que hacemos es generar los índices, en este caso podemos ver que en **Fig.30** realizamos el dado de cuatro caras con los índices mostrados para formar el una pirámide que este texturizada puesto que como podemos ver ya tenemos nuestras coordenadas de texturización que obtuvimos a partir de la imagen que vamos a utilizar. Finalmente simplemente se manda al CreateMesh que realizará la unión de los vértices para crear nuestra pirámide y se guardará en el meshList.

```
void Dado_4() {
    unsigned int dado4_indices[] = { 0,1,2, 3,4,5, 6,7,8, 9,10,11 };
    GLfloat dado4_vertices[] = {
        //Cara 1
        1.00f, 0.00f, 0.58f, 0.00f,0.00f, 0.00f, 1.00f, 0.00f, //0
        -1.00f, 0.00f, 0.58f, 1.00f,0.00f, 0.00f, 1.00f, 0.00f, //1
        0.00f, 0.00f,-1.15f, 0.50f,0.50f, 0.00f, 1.00f, 0.00f, //2
        //Cara 2
        1.00f, 0.00f, 0.58f, 1.00f,1.00f, 0.00f,-1.00f,-1.00f, //3
        -1.00f, 0.00f, 0.58f, 1.00f,0.00f, 0.00f,-1.00f,-1.00f, //4
        0.00f, 1.63f, 0.00f, 0.50f,0.50f, 0.00f,-1.00f,-1.00f, //5
        //Cara 3
        -1.00f, 0.00f, 0.58f, 0.00f,1.00f, 1.00f,-1.00f, 1.00f, //6
        0.00f, 0.00f,-1.15f, 1.00f,1.00f, 1.00f,-1.00f, 1.00f, //7
        0.00f, 1.63f, 0.00f, 0.50f,0.50f, 1.00f,-1.00f, 1.00f, //8
        //Cara 4
        1.00f, 0.00f, 0.58f, 0.00f,1.00f, -1.00f,-1.00f, 1.00f, //9
        0.00f, 0.00f,-1.15f, 0.00f,0.00f, -1.00f,-1.00f, 1.00f, //10
        0.00f, 1.63f, 0.00f, 0.50f,0.50f, -1.00f,-1.00f, 1.00f, //11
    };
    Mesh* dado4 = new Mesh();
    dado4->CreateMesh(dado4_vertices, dado4_indices, 96, 12);
    meshList.push_back(dado4);
}
```

Fig.30. Creación del dado de cuatro caras.

De la misma manera que creamos el dado de cuatro caras realizamos el de ocho a partir de polígonos que en este caso son triángulos, la única diferencia es claramente el número de índices declarados pues al ser más caras creamos una figura en forma de diamante por lo que la cantidad de índices y vértices es mayor. De igual forma vemos que tenemos las coordenadas de textura.

```

void Dado_8() {
    unsigned int dado8_indices[] = { 0,1,2, 3,4,5, 6,7,8, 9,10,11, 12,13,14, 15,16,17, 18,19,20, 21,22,23 };
    GLfloat dado8_vertices[] = {
        //Cara 1
        1.00f, 0.00f, 1.00f, 1.00f,0.50f, 0.00f,-1.00f, -1.00f,//0
        -1.00f, 0.00f, 1.00f, 0.00f,0.50f, 0.00f,-1.00f, -1.00f,//1
        0.00f, 1.51f, 0.00f, 0.50f,0.75f, 0.00f,-1.00f, -1.00f,//2
        //Cara 2
        -1.00f, 0.00f, 1.00f, 0.00f,0.50f, 1.00f,-1.00f, 0.00f,//3
        -1.00f, 0.00f,-1.00f, 0.00f,1.00f, 1.00f,-1.00f, 0.00f,//4
        0.00f, 1.51f, 0.00f, 0.50f,0.75f, 1.00f,-1.00f, 0.00f,//5
        //Cara 3
        -1.00f, 0.00f,-1.00f, 0.00f,1.00f, 0.00f,-1.00f, 1.00f,//6
        1.00f, 0.00f,-1.00f, 1.00f,1.00f, 0.00f,-1.00f, 1.00f,//7
        0.00f, 1.51f, 0.00f, 0.50f,0.75f, 0.00f,-1.00f, 1.00f,//8
        //Cara 4
        1.00f, 0.00f, 1.00f, 1.00f,0.50f, -1.00f,-1.00f, 0.00f,//9
        1.00f, 0.00f,-1.00f, 1.00f,1.00f, -1.00f,-1.00f, 0.00f,//10
        0.00f, 1.51f, 0.00f, 0.50f,0.75f, -1.00f,-1.00f, 0.00f,//11
        //Cara 5
        1.00f, 0.00f, 1.00f, 1.00f,0.50f, 0.00f, 1.00f,-1.00f,//12
        -1.00f, 0.00f, 1.00f, 0.00f,0.50f, 0.00f, 1.00f,-1.00f,//13
        0.00f,-1.51f, 0.00f, 0.50f,0.25f, 0.00f, 1.00f,-1.00f,//14
        //Cara 6
        -1.00f, 0.00f, 1.00f, 0.00f,0.50f, 1.00f, 1.00f, 0.00f,//15
        -1.00f, 0.00f,-1.00f, 0.00f,0.00f, 1.00f, 1.00f, 0.00f,//16
        0.00f,-1.51f, 0.00f, 0.50f,0.25f, 1.00f, 1.00f, 0.00f,//17
        //Cara 7
        -1.00f, 0.00f,-1.00f, 1.00f,0.00f, 0.00f, 1.00f, 1.00f,//18
        1.00f, 0.00f,-1.00f, 0.00f,0.00f, 0.00f, 1.00f, 1.00f,//19
        0.00f,-1.51f, 0.00f, 0.50f,0.25f, 0.00f, 1.00f, 1.00f,//20
        //Cara 8
        1.00f, 0.00f, 1.00f, 1.00f,0.50f, -1.00f, 1.00f, 0.00f,//21
        1.00f, 0.00f,-1.00f, 1.00f,0.00f, -1.00f, 1.00f, 0.00f,//22
        0.00f,-1.51f, 0.00f, 0.50f,0.25f, -1.00f, 1.00f, 0.00f,//23
    };
    Mesh* dado8 = new Mesh();
    dado8->CreateMesh(dado8_vertices, dado8_indices, 192, 24);
    meshList.push_back(dado8);
}

```

Fig.31. Creación del dado de ocho caras.

Dentro de nuestro main mandamos llamar nuestras funciones para la creación de dados, además de declarar e importar las texturas que se van a utilizar en ambos dados especificando la ruta donde se encuentran.

```

CreateObjects();    Casillas_Esquina();    Casillas_Inter();
Dado_4();           Dado_8();           Centro();
CreateShaders();

camera = Camera(glm::vec3(0.0f, 0.0f, 0.0f), glm::vec3(0.0f, 1.0f, 0.0f), -60.0f, 0.0f, 0.3f, 0.5f);

//Texturas para el piso
pisoTexture = Texture("Textures/piso.tga");           pisoTexture.LoadTextureA();
pisoTexture2 = Texture("Textures/CentroTableroSolo.tga");   pisoTexture2.LoadTextureA();
CentroTexture = Texture("Textures/CentroTablero.tga");       CentroTexture.LoadTextureA();

//Texturas de los Dados
Dado4_1 = Texture("Textures/Dado4_1.tga");           Dado4_1.LoadTextureA();
Dado4_2 = Texture("Textures/Dado4_2.tga");           Dado4_2.LoadTextureA();
Dado8 = Texture("Textures/Dado8.tga");               Dado8.LoadTextureA();

```

Fig.32. Importar texturas de los dados.

Inicializamos las variables que vamos a utilizar para lograr la rotación de los dados.

```
// Variables para rotacion de los Dados.
movDado = 0.0f;      movOffsetDado = 0.8f;
RotacionX4, RotacionZ4 = 0.0f; RotacionX8, RotacionZ8 = 0.0f;
DadoRotando = true;   DadoPiso = false;
```

Fig.33. Inicialización variables de rotación de dados.

Para realizar la rotación de las respectivas caras del dado a partir de un valor aleatorio al tirar los dados entramos en un if en donde a utilizando la función rand() asignamos a ambos dados un número aleatorio que será el índice en un arreglo con las distintas caras pertenecientes a cada dado y con esto es que elegimos una de esas caras dentro del arreglo.

```
T_Transcurrido += deltaTime;
if (T_Transcurrido >= TIEMPO_CAMBIO) { FASE_01 = !FASE_01; T_Transcurrido = 0.0f; }

float angle = (T_Transcurrido / TIEMPO_CAMBIO) * 3.14159f;

// Calcular la dirección basada en el ángulo
float xDir = cos(angle);

//-- Movimiento para el tiro de los Dados --
if (mainWindow.gettiro() == true && DadoRotando == true) {
    movDado -= movOffsetDado * deltaTime;

    int sec_D4 = rand() % 4; //Giro para el Dado 4.
    Giro4 rot4 = Vista_Caras4[sec_D4];
    RotacionX4 = rot4.x; RotacionZ4 = rot4.z;
    mainWindow.setUltimoValorDado4(rot4.valor);

    int sec_D8 = rand() % 8; //Giro para el Dado 8.
    Giro8 rot8 = Vista_Caras8[sec_D8];
    RotacionX8 = rot8.x; RotacionZ8 = rot8.z;
    mainWindow.setUltimoValorDado8(rot8.valor);
}
```

Fig.34. Asignación de rotación aleatoria de los dados.

Luego de hacer la rotación de los dados al tirar, realizamos la suma de los valores que obtuvimos para que esa sea la cantidad de casillas que debemos avanzar, esto verificando que ya se llegó al piso en la tirada entonces guardamos en avance la cantidad que obtuvimos de cada cara de los dados al tirar puesto que serán las casillas que nos vamos a mover.

Con el do-while es que realizamos el movimiento mientras disminuimos el avance hasta que sea igual a cero pues ya llegaríamos a la casilla correspondiente, y aumentamos la posición ya que el modelo que irá desplazando dependiendo de su correspondiente arreglo de casillas asignadas al tirar los dados.

```

if (movDado < -24.0f) {
    DadoRotando = false;
    DadoPiso = true;
    PuedeSubir = false;
    avance = rot4.valor + rot8.valor;
    printf("\n - - - - - \n Los valores de los dados en el ultimo tiro fueron : \n");
    printf("Valor del Dado4: %.0f\n", rot4.valor);
    printf("Valor del Dado8: %.0f\n", rot8.valor);
    printf("Numero de Casillas a mover: %.0f\n", avance);

    do {
        if (Posicion == 40) { Posicion = 0; }
        PCasillas movcasilla = Mov_Casillas[Posicion];
        new_pos += movcasilla.mov;
        strcpy_s(casilla_Activacion, sizeof(casilla_Activacion), movcasilla.activacion); // Versión segura // Usar strcpy para copiar el string
        Posicion++; avance--;
        printf("Sumando, valor nuevo = %f\nEstamos en el arreglo = %d, de valor = %f\n", new_pos, Posicion, movcasilla.mov);
        printf("Nombre de la casilla: %s\n", casilla_Activacion);
    } while (avance != 0);
    printf("\n PROCEDEMOS CON EL MOVIMIENTO \n");
    Continuar = true;
}

if (mainWindow.getreinicio() == true && DadoPiso == true) { movDado = 0.0f; DadoRotando = true; DadoPiso = false; }

```

Fig.35. Ciclo DO WHILE para calcular el valor total de casillas a moverse.

→ El avatar caminará sobre el tablero la cantidad de casillas equivalente al número obtenido con los dados.

Para esta actividad declaramos diversas variables que nos ayudan a controlar el movimiento y la posición en la que se encuentra el personaje, movx, movz y giro que son variables para controlar el movimiento y la rotación del personaje.

```

// Variables para movimiento en el Tablero.
int Posicion;
float movX, movZ, giro, avance;
float movOffsetHeli, V_Rot_Helice;
float UbiCesq1, UbiCesq2, new_pos;
bool Cas1, Cas2, Cas3, Cas4, Continuar, PuedeSubir;
char casilla_Activacion[3];

bool PerPrincipal1, PerPrincipal2, PerPrincipal3;

```

Fig.36. Variables para movimiento de personajes en el tablero .

```

// Variables para movimiento en el Tablero.
Posicion = 0;
movX, movZ, giro, avance = 0.0f;
movOffsetHeli = 0.5f; V_Rot_Helice = 0.0f;
UbiCesq1 = 54; UbiCesq2 = 54; new_pos = 0;
Cas1 = true; Cas2 = false; Cas3 = false; Cas4 = false; PuedeSubir = false;
PerPrincipal1 = true; PerPrincipal2 = false; PerPrincipal3 = false;

```

Fig.37. Inicialización de variables para controlar el movimiento del personaje.

Para el movimiento más controlado del personaje creamos la estructura PCasillas, la cual nos ayuda a identificar cada una de las casillas en un arreglo de 40 duplas (valor y nombre) las cuales nos ayudan al momento de hacer avanzar a nuestro personaje ya que el primer valor es 0.

```

struct PCasillas { float mov; char activacion[3]; };
PCasillas Mov_Casillas[40] = {
    {7.0f, "A1"}, /*A1*/ {5.0f, "A2"}, /*A2*/ {5.0f, "A3"}, /*A3*/ {5.0f, "A4"}, /*A4*/ {5.0f, "A5"}, /*A5*/
    {5.0f, "A6"}, /*A6*/ {5.0f, "A7"}, /*A7*/ {5.0f, "A8"}, /*A8*/ {5.0f, "A9"}, /*A9*/ {7.0f, "S1"}, /*S2*/

    {7.0f, "B1"}, /*B1*/ {5.0f, "B2"}, /*B2*/ {5.0f, "B3"}, /*B3*/ {5.0f, "B4"}, /*B4*/ {5.0f, "B5"}, /*B5*/
    {5.0f, "B6"}, /*B6*/ {5.0f, "B7"}, /*B7*/ {5.0f, "B8"}, /*B8*/ {5.0f, "B9"}, /*B9*/ {7.0f, "S2"}, /*S3*/

    {7.0f, "C1"}, /*C1*/ {5.0f, "C2"}, /*C2*/ {5.0f, "C3"}, /*C3*/ {5.0f, "C4"}, /*C4*/ {5.0f, "C5"}, /*C5*/
    {5.0f, "C6"}, /*C6*/ {5.0f, "C7"}, /*C7*/ {5.0f, "C8"}, /*C8*/ {5.0f, "C9"}, /*C9*/ {7.0f, "S3"}, /*S4*/

    {7.0f, "D1"}, /*D1*/ {5.0f, "D2"}, /*D2*/ {5.0f, "D3"}, /*D3*/ {5.0f, "D4"}, /*D4*/ {5.0f, "D5"}, /*D5*/
    {5.0f, "D6"}, /*D6*/ {5.0f, "D7"}, /*D7*/ {5.0f, "D8"}, /*D8*/ {5.0f, "D9"}, /*D9*/ {7.0f, "S4"}, /*S1*/ };

```

Fig.38. Estructura para indicar controlar las unidades de movimiento.

El funcionamiento del código es el siguiente: una vez que se ha calculado la distancia que debe recorrer el personaje se ejecuta una serie de ifs, cada uno indicando si la posición del personaje está antes de la casilla de esquina 1,2, 3 y 4, si se encuentra en la posición de la primer esquina se verifica si esta es mayor a la cantidad que debía moverse, si ese es el caso activa la variable para que el modelo ejecute su animación de subir y bajar, en caso contrario, indicando que aún no llega a la posición que debería tener de acuerdo al valor de los dados, incrementa el valor de la variable del eje movZ, y ahí mismo mientras incrementa verifica que su distancia no sea mayor a 54, ya que esto indicaría que llegó a la casilla de la esquina, si ese es el caso cambia el valor de la casilla actual de esquina a la siguiente, reinicia sus variables de posición y manda el valor de giro del personajes de menos 90°, lo que prepara al personaje para la siguiente casilla. Las demás secuencias de else if realizan el mismo procedimiento pero el movimiento es ajustado a la dirección en la que se debe mover el Avatar, para el caso de la primera esquina nos dirigimos hacia Z positivo, para el caso de la segunda esquina nos dirigimos hacia X negativa, en el caso de la tercera esquina nos dirigimos hacia Z negativo y en el caso de llegar a la cuarta esquina, qué es la casilla GO nuevamente, también asigna el valor a unas banderas que funcionan para el cambio de personaje en cada recorrido completo del tablero.

```

if (Continuar = true) {
    if (Cas1 == true) {
        if (movZ >= new_pos) {PuedeSubir = true; }
        if (movZ < new_pos) {
            movZ += movOffsetHeli * deltaTime;
            if (movZ > 54) {
                Cas1 = false; Cas2 = true;
                new_pos = new_pos - 54;
                PuedeSubir = true;
                giro -= 90.0f;
                printf("\n C1Valor new_pos: %f\n", new_pos);
            }
        }
    }
    else if (Cas2 == true) {
        if (movX <= (-new_pos)) { PuedeSubir = true; }
        if (movX > (-new_pos)) {
            movX -= movOffsetHeli * deltaTime;
            if (movX < -54) {
                Cas2 = false; Cas3 = true;
                new_pos = new_pos - 54;
                PuedeSubir = true;
                giro -= 90.0f;
                printf("\n C2Valor new_pos: %f\n", new_pos);
            }
        }
    }
}

```

Fig.39. Secuencia de ifs para el movimiento dependiendo el lado del tablero en el que se encuentra el personaje.

```

else if (Cas3 == true) {
    if (movZ <= (-new_pos + UbiCesq1)) { PuedeSubir = true; }
    if (movZ > (-new_pos + UbiCesq1)) {
        movZ -= movOffsetHeli * deltaTime;
        if (movZ < 1) {
            Cas3 = false; Cas4 = true;
            new_pos = new_pos - 54;
            PuedeSubir = true;
            giro -= 90.0f;
            printf("\n C3Valor new_pos: %f\n", new_pos);
        }
    }
    else if (movZ == (-new_pos + UbiCesq1)) { UbiCesq1 = movZ; }
}
else if (Cas4 == true) {
    if (movX >= (new_pos - UbiCesq2)) { PuedeSubir = true; }
    if (movX < (new_pos - UbiCesq2)) {
        movX += movOffsetHeli * deltaTime;
        if (movX > -1) {
            Cas4 = false; Cas1 = true; Cas2 = false; Cas3 = false;
            new_pos = new_pos - 54;
            PuedeSubir = true;
            giro -= 90.0f;
            printf("\n C4Valor new_pos: %f\n", new_pos);
            if (PerPrincipal1 == true) { PerPrincipal1 = false; PerPrincipal2 = true; PerPrincipal3 = false; }
            else if (PerPrincipal2 == true) { PerPrincipal1 = false; PerPrincipal2 = false; PerPrincipal3 = true; }
            else if (PerPrincipal3 == true) { PerPrincipal1 = true; PerPrincipal2 = false; PerPrincipal3 = false; }
        }
    }
    else if (movX == (new_pos - UbiCesq2)) { UbiCesq2 = movX; }
}
}

```

Fig.40. Secuencia de ifs para el movimiento dependiendo el lado del tablero en el que se encuentra el personaje.

Ya con la indicación de movimiento hacia el eje que se debe mover el Avatar se asignan estas variables de movimiento y de giro a la asignación del personaje principal del tablero

```

model = glm::mat4(1.0f); // Personaje Principal en el tablero
model = glm::translate(model, glm::vec3(movX, 0.0f, movZ));
model = glm::rotate(model, glm::radians(giro), glm::vec3(0.0f, 1.0f, 0.0f));
if (PerPrincipal1 == true) {
    model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
    modelaux2 = model;
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    Perry.RenderModel(); }
else if (PerPrincipal2 == true) {
    model = glm::scale(model, glm::vec3(0.6f, 0.6f, 0.6f));
    modelaux2 = model;
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    Bombon.RenderModel(); }

```

Fig.41. Asignación de movimiento del personaje principal de acuerdo a la distancia calculada.

La ciudad:

→ Alrededor del tablero habrá elementos adicionales de los universos de los personajes elegidos por los alumnos integrantes del equipo para que visualmente no se vea que el tablero del juego de mesa está flotando en el espacio.

Lo primero es la declaración de las variables de tipo Model que nos ayudarán a cargar en ellas los modelos correspondientes, asimismo declaramos la ubicación de los archivos de los modelos.

```

// Modelos alrededor del tablero
Model Gato, Octi, Arbol, Burbuja, Bellota, Mojo, Peludito, Princesa, Profesor, Sedusa, Casa, Edificio, Alcantaria, Phineas, Ornitorincio,
Isabella, PeterPanda, Doofenshmirtz, TerryTortuga, Candace, Doofenshmirtz2D, Buford, PinkyChihuahua, Carl, Ferb,
HiroHamada_M, Planta1_M, AutoChico_M, GogoTomago_M, Motocicleta_M, GoldenGate_M, Planta2_M, BaymaxSuperArmadura_M, AutoGrande_M,
Fred_M, Planta3_M, CasaHiro_M, Megabot_M, Lampara, Lampara2, esf;

// Modelos ciudad
Model Edificio1, Edificio2, Edificio3, Edificio4, Edificio5, Edificio6;
Model Casa2, Doof, Edificio7, Edificio8, Edificio9, Edificio10, Edificio11, Edificio12, Edificio13, Edificio14;
Model Edificio15, Edificio16, Edificio17, Edificio18, Torre, Puente;

```

Fig.42. Variables para los modelos

```

// Los 40 Modelos en el Tablero
Perry = Model();
Bombon = Model();
BaymaxBlanco_M = Model();
esf = Model();

Gato = Model();
Octi = Model();
Arbol = Model();
Burbuja = Model();
Bellota = Model();
Mojo = Model();
Peludito = Model();
Princesa = Model();
Profesor = Model();
Sedusa = Model();
Casa = Model();
Edificio = Model();
Alcantaria = Model();
HiroHamada_M = Model();
Planta1_M = Model();
AutoChico_M = Model();
GogoTomago_M = Model();
Motocicleta_M = Model();
GoldenGate_M = Model();
Planta2_M = Model();
BaymaxSuperArmadura_M = Model();

Perry.LoadModel("Models/Perry.obj");
Bombon.LoadModel("Models/bombon.obj");
BaymaxBlanco_M.LoadModel("Models/BaymaxBlanco/Bigmax_White_OBJ.obj");
esf.LoadModel("Models/esfera.obj");

Gato.LoadModel("Models/gato.obj");
Octi.LoadModel("Models/octi.obj");
Arbol.LoadModel("Models/arbol.obj");
Burbuja.LoadModel("Models/burbuja.obj");
Bellota.LoadModel("Models/bellota.obj");
Mojo.LoadModel("Models/mojo.obj");
Peludito.LoadModel("Models/peludito.obj");
Princesa.LoadModel("Models/princesa.obj");
Profesor.LoadModel("Models/profesor.obj");
Sedusa.LoadModel("Models/sedusa.obj");
Casa.LoadModel("Models/casa.obj");
Edificio.LoadModel("Models/edificio.obj");
Alcantaria.LoadModel("Models/alcantaria.obj");
HiroHamada_M.LoadModel("Models/HiroHamada/hiro.obj");
Planta1_M.LoadModel("Models/Planta1/Phormium_1.obj");
AutoChico_M.LoadModel("Models/AutoChico/isetta.obj");
GogoTomago_M.LoadModel("Models/GogoTomago/gogo.obj");
Motocicleta_M.LoadModel("Models/Motocicleta/motocicleta.obj");
GoldenGate_M.LoadModel("Models/GoldenGate/golden gate bridge.obj");
Planta2_M.LoadModel("Models/Planta2/hoewa_Forsteriana_1.obj");
BaymaxSuperArmadura_M.LoadModel("Models/BaymaxSuperArmadura/baymax_super.obj");

```

Fig.43. Modelos cargados de los personajes.

```

AutoGrande_M = Model();
Fred_M = Model();
Planta3_M = Model();
CasaHiro_M = Model();
Megabot_M = Model();
Phineas = Model();
Ornitocyborg = Model();
Isabella = Model();
PeterPanda = Model();
Doofenshmirtz = Model();
TerryTortuga = Model();
Candace = Model();
Doofenshmirtz2D = Model();
Buford = Model();
PinkyChihuahua = Model();
Carl = Model();
Ferb = Model();
Lampara = Model();
Lampara2 = Model();

AutoGrande_M.LoadModel("Models/AutoGrande/500_fiat_lod_a.obj");
Fred_M.LoadModel("Models/Fred/fred.obj");
Planta3_M.LoadModel("Models/Planta3/Planta3.obj");
CasaHiro_M.LoadModel("Models/CasaHiro/CasaHiro.obj");
Megabot_M.LoadModel("Models/Megabot/MegaBot.obj");
Phineas.LoadModel("Models/PHINEAS.obj");
Ornitocyborg.LoadModel("Models/OrnitoCyborg.obj");
Isabella.LoadModel("Models/Isabella.obj");
PeterPanda.LoadModel("Models/PeterPanda.obj");
Doofenshmirtz.LoadModel("Models/Doofenshmirtz.obj");
TerryTortuga.LoadModel("Models/TerryTortuga.obj");
Candace.LoadModel("Models/Candace.obj");
Doofenshmirtz2D.LoadModel("Models/Doofenshmirtz2D.obj");
Buford.LoadModel("Models/Buford.obj");
PinkyChihuahua.LoadModel("Models/PinkyChihuahua.obj");
Carl.LoadModel("Models/Carl.obj");
Ferb.LoadModel("Models/Ferb.obj");
Lampara.LoadModel("Models/fobj_lamp_MODFIC.obj");
Lampara2.LoadModel("Models/obj_0401_street_lamp.obj");

```

Fig.44. Modelos cargados de los personajes.

De igual manera para cada uno de los edificios de alrededor del tablero que representan la ciudad se le carga el modelo a la variable para posteriormente sean utilizados con las transformaciones geométricas correspondientes.

```

// Edificios Ciudad
Edificio1 = Model();
Edificio1.LoadModel("Models/edificio1.obj");
Edificio2 = Model();
Edificio2.LoadModel("Models/edificio2.obj");
Edificio3 = Model();
Edificio3.LoadModel("Models/edificio3.obj");
Edificio4 = Model();
Edificio4.LoadModel("Models/edificio4.obj");
Edificio5 = Model();
Edificio5.LoadModel("Models/edificio5.obj");
Edificio6 = Model();
Edificio6.LoadModel("Models/edificio6.obj");

//Phineas and Ferb
Casa2 = Model();
Casa2.LoadModel("Models/casa2.obj");
Doof = Model();
Doof.LoadModel("Models/doof.obj");
Edificio7 = Model();
Edificio7.LoadModel("Models/edificio7.obj");
Edificio8 = Model();
Edificio8.LoadModel("Models/edificio8.obj");

Edificio9 = Model();
Edificio9.LoadModel("Models/edificio9.obj");
Edificio10 = Model();
Edificio10.LoadModel("Models/edificio10.obj");

//BIG HERO SIX
Edificio15 = Model();
Edificio15.LoadModel("Models/edificio15.obj");
Edificio16 = Model();
Edificio16.LoadModel("Models/edificio16.obj");
Edificio17 = Model();
Edificio17.LoadModel("Models/edificio17.obj");
Edificio18 = Model();
Edificio18.LoadModel("Models/edificio18.obj");
Torre = Model();
Torre.LoadModel("Models/torre.obj");
Puente = Model();
Puente.LoadModel("Models/puente.obj");

```

Fig.44 y Fig.45 . Modelos cargados de los edificios.

Al hablar de transformaciones nos referimos a las siguientes, donde a cada modelo de la ciudad se le asigna una traslación, una rotación y una escala de ser necesario para poder acomodar cada elemento en un lugar adecuado, que no interfiera con ningún otro modelo y que todos tengan un tamaño similar.

```

//Edificio1
model = modelaux;
model = glm::translate(model, glm::vec3(92.0f, -0.8f, 30.0f));
//model = glm::translate(model, glm::vec3(42.0f, -0.8f, -20.0f));
model = glm::rotate(model, -90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(1.0f, 1.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Edificio1.RenderModel();

//Edificio2
model = modelaux;
model = glm::translate(model, glm::vec3(85.0f, -0.8f, -20.0f));
//model = glm::translate(model, -90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(1.0f, 1.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Edificio2.RenderModel();

//Alcaldia
model = modelaux;
model = glm::translate(model, glm::vec3(52.0f, -0.8f, 30.0f));
model = glm::rotate(model, -90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(3.0f, 3.0f, 3.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Alcaldia.RenderModel();

```

Fig.46 . Ubicación de los modelos de edificio de la ciudad de saltadilla.

```

//PHINEAS Y FERB
//Casa
model = modelaux;
model = glm::translate(model, glm::vec3(-23.0f, -0.8f, -45.0f));
model = glm::scale(model, glm::vec3(0.7f, 0.7f, 0.7f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Casa2.RenderModel();

//Doof casa
model = modelaux;
model = glm::translate(model, glm::vec3(-90.0f, -0.8f, -60.0f));
model = glm::rotate(model, 90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.7f, 0.7f, 0.7f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Doof.RenderModel();

//Edificio7
model = modelaux;
model = glm::translate(model, glm::vec3(-85.0f, -0.8f, 5.0f));
model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Edificio7.RenderModel();

//Edificio8
model = modelaux;
model = glm::translate(model, glm::vec3(-87.0f, -0.8f, -25.0f));
model = glm::rotate(model, 90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.6f, 0.6f, 0.6f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Edificio8.RenderModel();

```

Fig.47 . Ubicación de los modelos de edificio del Área limítrofe.

```

// --- BIG HERO SIX -----
//Edificio15
model = modelaux;
model = glm::translate(model, glm::vec3(-35.0f, -0.8f, 80.0f));
model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.8f, 0.8f, 0.8f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Edificio15.RenderModel();

//Edificio16
model = modelaux;
model = glm::translate(model, glm::vec3(-57.0f, -0.8f, 80.0f));
model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.4f, 0.4f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Edificio16.RenderModel();

//Torre
model = modelaux;
model = glm::translate(model, glm::vec3(-87.0f, -0.8f, 30.0f));
model = glm::scale(model, glm::vec3(0.7f, 0.7f, 0.7f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Torre.RenderModel();

//Edificio18
model = modelaux;
model = glm::translate(model, glm::vec3(-87.0f, -0.8f, 55.0f));
model = glm::rotate(model, 90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Edificio18.RenderModel();

```

Fig.48. Ubicación de los modelos de edificio de San Fran Tokyo.

Iluminación:

→ Se tendrá una luz de forma puntual que irá iluminando al avatar durante el ciclo de noche y estará apagada durante el ciclo de día.

Para este procedimiento declaramos un point light, después declaramos un modelo de esfera en el que colocaremos la point light, ajustada en posición un poco atrás del avatar y asignándole las variables de giro, no es necesario indicar las variables de movimiento ya que se encuentra en jerarquía con el modelo del personaje principal por lo tanto lo seguirá en su recorrido, algo que se declaró extra es que si el modelo es de baymax, la luz aumenta su posición para que sea visible detrás del personaje, para el caso de los otros 2 personajes bombón y Perry la ubicación de la luz es más abajo

```

pointLights[1] = PointLight(1.0f, 1.0f, 1.0f, 3.0f, 1.0f, 0.0f, 0.0f, 0.0f, 1.0f, 0.2f, 0.1f);
pointLightCount++; // LUZ DE SEGUIMIENTO.

```

Fig.49. Declaración de la luz de seguimiento.

```

if (!FASE_01) {
    model = modelaux2; //Luz de seguimiento
    if (PerPrincipal3 == true) {
        model = glm::translate(model, glm::vec3(-20.0f, 70.0f, -20.0f));
        model = glm::rotate(model, glm::radians(giro), glm::vec3(0.0f, 1.0f, 0.0f));
        model = glm::scale(model, glm::vec3(0.8f, 0.8f, 0.8f));
    }
    else {
        model = glm::translate(model, glm::vec3(-4.0f, 11.0f, -4.0f));
        model = glm::rotate(model, glm::radians(giro), glm::vec3(0.0f, 1.0f, 0.0f));
        model = glm::scale(model, glm::vec3(0.1f, 0.1f, 0.1f));
    }
    modelaux2 = model;
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    esf.RenderModel();
    glm::vec3 lghtPrinc = glm::vec3(modelaux2[3]);
    pointLights[1].SetPosition(lghtPrinc.x, lghtPrinc.y, lghtPrinc.z);
    activeLights[activeLightCount++] = pointLights[1];
}

```

Fig.50. Declaración de la luz de seguimiento.

→ Se tendrán en cada esquina del tablero 4 fuentes de luz entre puntual y posicional que se prenderán durante el ciclo de noche si el avatar pasa por esa casilla.

Para esta implementación escogimos 3 spotlights para las luces y un point light para la última, además declaramos algunos modelos de lámparas de calle para implementar mejor estas luces, la declaración de los modelos es la misma acompañada de la posición específica del point light o spotlight, para que esto se encienda correctamente declaramos un if para encendido y un else para apagado en cada uno, hola en el caso del point light simplemente indicamos que se manda una vez la información de encendido cuándo es de noche a través de la bandera FASE_01, mientras que en los spotlights tenemos que indicar el mandado de información para el encendido pero para el apagado nos envía nada.

```
unsigned int pointLightCount = 0;
pointLights[0] = PointLight(1.0f, 1.0f, 1.0f, 5.0f, 3.0f, 0.0f, 0.0f, 0.0f, 1.0f, 0.2f, 0.1f);
pointLightCount++; // LUZ ESQUINA 01.

pointLights[1] = PointLight(1.0f, 1.0f, 1.0f, 3.0f, 1.0f, 0.0f, 0.0f, 0.0f, 1.0f, 0.2f, 0.1f);
pointLightCount++; // LUZ DE SEGUIMIENTO.

unsigned int spotLightCount = 0;

spotLights[0] = Spotlight(1.0f, 1.0f, 1.0f, 4.0f, 1.0f, 0.0f, 0.0f, 0.0f, 1.0f, 0.0f, 0.0f, 1.0f, 0.1f, 0.05f, 45.0f);
spotLightCount++; // LUZ ESQUINA 02.

spotLights[1] = Spotlight(1.0f, 1.0f, 1.0f, 4.0f, 1.0f, 0.0f, 0.0f, 0.0f, 1.0f, 0.0f, 0.0f, 1.0f, 0.1f, 0.05f, 45.0f);
spotLightCount++; // LUZ ESQUINA 03.

spotLights[2] = Spotlight(1.0f, 1.0f, 1.0f, 4.0f, 1.0f, 0.0f, 0.0f, 0.0f, 1.0f, 0.0f, 0.0f, 1.0f, 0.1f, 0.05f, 45.0f);
spotLightCount++; // LUZ ESQUINA 04.
```

Fig.51. Arreglos de luces puntuales y spot.

```
//Lampara 01
model = modelLamp1;
model = glm::translate(model, glm::vec3(8.0f, 0.2f, -8.0f));
model = glm::scale(model, glm::vec3(0.15f, 0.15f, 0.15f));
modelLamp1 = model;
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Lampara.RenderModel();
glm::vec3 lamparaPos1 = glm::vec3(modelLamp1[3]);
pointLights[0].SetPosition(lamparaPos1.x, lamparaPos1.y + 8.2f, lamparaPos1.z);
if (!FASE_01) { activeLights[activeLightCount++] = pointLights[0]; }

//Lampara 02
model = modelLamp2;
model = glm::translate(model, glm::vec3(6.0f, 0.2f, 62.0f));
model = glm::rotate(model, -135 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.25f, 0.25f, 0.25f));
modelLamp2 = model;
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Lampara2.RenderModel();
glm::vec3 lamparaPos2 = glm::vec3(modelLamp2[3]);
glm::vec3 lightPos = glm::vec3(lamparaPos2.x - 3.2f, lamparaPos2.y + 11.6f, lamparaPos2.z - 3.2f);
if (!FASE_01) { spotLights[0].SetFlash(lightPos, glm::vec3(0.0f, -1.0f, 0.0f)); } // Noche - Luz encendida
else { spotLights[0].SetFlash(lightPos, glm::vec3(0.0f)); } // Día - Luz apagada
```

Fig.52. Posición de las lámparas y ciclo de encendido y apagado.

```

//Lampara 04
model = modelLamp4;
model = glm::translate(model, glm::vec3(-62.0f, 0.2f, -6.0f));
model = glm::rotate(model, 45 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.25f, 0.25f, 0.25f));
modelLamp4 = model;
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Lampara2.RenderModel();
glm::vec3 lamparaPos4 = glm::vec3(modelLamp4[3]);
glm::vec3 lightPos3 = glm::vec3(lamparaPos4.x + 3.2f, lamparaPos4.y + 11.6f, lamparaPos4.z + 3.2f);
if (!FASE_01) { spotLights[2].SetFlash(lightPos3, glm::vec3(0.0f, -1.0f, 0.0f)); } // Noche - Luz encendida
else { spotLights[2].SetFlash(lightPos3, glm::vec3(0.0f)); } // Dia - Luz apagada

shaderList[0].SetPointLights(activeLights, activeLightCount);

```

Fig.53. Posición de las lámparas y ciclo de encendido y apagado.

→ Se tendrá una luz de tipo direccional que representará a un sol la cual se irá “moviendo” en el escenario en dirección Este-Oeste

Para iniciar con esta declaración primero generamos las 2 luces hola qué representan el día y la noche, hoy después generamos una secuencia para medir el tiempo transcurrido en este caso dando un total de 16 segundos aproximadamente para el cambio entre el día y la noche hoy además para tener un mejor efecto en el movimiento del sol se calcula el ángulo de dirección en el eje x que será el movimiento del sol, siendo una función coseno a partir hoy de los tiempos transcurridos y el tiempo de cambio, pero también multiplicado por pi para darle un mejor movimiento

```

LightDAY = DirectionalLight (1.0f, 1.0f, 0.9f, 0.4f, 0.6f, 0.0f, -1.0f, 0.0f);
LightNIGHT = DirectionalLight (0.8f, 0.8f, 1.0f, 0.1f, 0.2f, 0.0f, -1.0f, 0.0f);

```

Fig.54. Declaración de Luces de día y de Noche.

```

T_Transcurrido += deltaTime;
if (T_Transcurrido >= TIEMPO_CAMBIO) { FASE_01 = !FASE_01; T_Transcurrido = 0.0f; }

float angle = (T_Transcurrido / TIEMPO_CAMBIO) * 3.14159f;

// Calcular la dirección basada en el ángulo
float xDir = cos(angle);

```

Fig.55. Cálculo del ciclo de luz de día y de noche.

Así mismo una vez transcurrido el tiempo se activa en true la bandera FASE_01, hola qué indica que la luz desde día, en caso contrario la iluminaciones de noche, para controlar este cambio se declaró un if que se ejecuta a partir del valor de esta bandera, en ambos casos el valor que calculamos de dirección en X se coloca, realmente la única diferencia entre ambas declaraciones es la luz puntual y el skybox que cambia con esta.

```

if (FASE_01) {
    LightDAY.SetDirection(xDir, -0.25f, 0.0f/*zDir*/);
    shaderList[0].SetDirectionalLight(&LightDAY);
    skyboxDia.DrawSkybox(camera.calculateViewMatrix(), projection);
}
else {
    LightNIGHT.SetDirection(xDir, -0.25f, 0.0f/*zDir*/);
    shaderList[0].SetDirectionalLight(&LightNIGHT);
    skyboxNoche.DrawSkybox(camera.calculateViewMatrix(), projection);
}

```

Fig.56. Secuencia de iluminado de día o noche.

Animación:

- Alrededor del tablero en lo que es la ciudad estarán los otros avatares de los integrantes del equipo haciendo una animación característica del personaje (No puede ser animación de caminata)

Puntos Adicionales:

Cambio de avatar en el tablero, Si se implementa que cada que el avatar cruce por la casilla de inicio (una esquina indicada) se cambia por uno de los avatares de los otros integrantes del equipo.

Para implementar este cambio de avatar, creamos una secuencia de else if para cambiar la posición en cada asignación de personajes, todo a través de las banderas PerPrincipal, a las cuales se les asigna un valor durante la última secuencia de movimiento del personaje, la secuencia para asignar los valores a cada bandera dependen de la que esté activa primero, que será la de PerPrincipal1, de de perry.

El elemento que cuenta con más declaraciones es Baymax, ya que fue el único personaje que realiza una animación mientras está afuera del tablero.

```

else if (Cas4 == true) {
    if (movX >= (new_pos - UbiCesq2)) { PuedeSubir = true; }
    if (movX < (new_pos - UbiCesq2)) {
        movX += movOffsetHeli * deltaTime;
        if (movX > -1) {
            Cas4 = false; Cas1 = true; Cas2 = false; Cas3 = false;
            new_pos = new_pos - 54;
            PuedeSubir = true;
            giro -= 90.0f;
            printf("\n C4Valor new_pos: %f\n", new_pos);
            if (PerPrincipal1 == true) { PerPrincipal1 = false; PerPrincipal2 = true; PerPrincipal3 = false; }
            else if (PerPrincipal2 == true) { PerPrincipal1 = false; PerPrincipal2 = false; PerPrincipal3 = true; }
            else if (PerPrincipal3 == true) { PerPrincipal1 = true; PerPrincipal2 = false; PerPrincipal3 = false; }
        }
    }
    else if (movX == (new_pos - UbiCesq2)) { UbiCesq2 = movX; }
}

```

Fig.57. Activación de banderas para el cambio de posición de avatar.

De acuerdo con las banderas antes mencionadas es posible que cada que una de estas se activa el modelo de cada uno de los personajes principales: Perry el Ornitorrinco, Bombon y Baymax se intercambien en la posición inicial del tablero hasta que, después de cruzar la casilla GO, el siguiente modelo en la lista sea el que simule las tiradas.

```

model = glm::mat4(1.0f); // Personaje Principal en el tablero
model = glm::translate(model, glm::vec3(movX, 0.0f, movZ));
model = glm::rotate(model, glm::radians(giro), glm::vec3(0.0f, 1.0f, 0.0f));
if (PerPrincipal1 == true) {
    model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
    modelaux2 = model;
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    Perry.RenderModel(); }
else if (PerPrincipal2 == true) {
    model = glm::scale(model, glm::vec3(0.6f, 0.6f, 0.6f));
    modelaux2 = model;
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    Bombon.RenderModel(); }

```

Fig.58. Impresión de los modelos de acuerdo a las banderas en el tablero.

```

else if (PerPrincipal3 == true) {
    model = glm::mat4(1.0); // Baymax Blanco Con Jerarquía
    model = glm::translate(model, glm::vec3(movX, 0.0f, movZ));
    model = glm::rotate(model, glm::radians(giro), glm::vec3(0.0f, 1.0f, 0.0f));
    model = glm::scale(model, glm::vec3(0.1f, 0.1f, 0.1f));
    modelaux2 = model;
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    BaymaxBlanco_M.RenderModel();

    model = modelaux2;
    model = glm::translate(model, glm::vec3(-12.0f, 60.0f, 0.0f));
    modelaux2 = model;
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    BrazoDerechoBaymax.RenderModel();

    model = modelaux2;
    model = glm::translate(model, glm::vec3(-13.0f, -30.0f, 0.0f));
    modelaux2 = model;
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    ManoBaymax.RenderModel();

    model = modelaux2;
    model = glm::translate(model, glm::vec3(-5.0f, -5.0f, 0.0f));
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    DedoMedioBaymax.RenderModel();

    model = modelaux2;
    model = glm::translate(model, glm::vec3(-5.0f, -4.0f, 2.0f));
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    DedoIzquierdoBaymax.RenderModel();

    model = modelaux2;
    model = glm::translate(model, glm::vec3(-5.0f, -4.0f, -2.0f));
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    DedoDerechoBaymax.RenderModel();

    model = modelaux2;
    model = glm::translate(model, glm::vec3(2.0f, -4.0f, 2.0f));
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    DedoGordoBaymax.RenderModel();
}

```

Fig.59. Impresión de modelo Baymax de acuerdo a las banderas en el tablero.

```

model = modelaux; // Personaje Secundario1 afuera del tablero
model = glm::translate(model, glm::vec3(8.0f, -0.8f, 0.0f));
model = glm::rotate(model, -90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));

if (PerPrincipal1 == true) {
    model = glm::scale(model, glm::vec3(0.6f, 0.6f, 0.6f));
    modelaux2 = model;
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    Bombon.RenderModel();
}

else if (PerPrincipal2 == true) {
    model = glm::mat4(1.0); // Baymax Animado
    model = glm::translate(model, glm::vec3(8.0f, -0.8f, 0.0f));
    model = glm::rotate(model, -90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
    model = glm::scale(model, glm::vec3(0.1f, 0.1f, 0.1f));
    modelauxBaymax = model;
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    BaymaxBlanco_M.RenderModel();

    model = modelauxBaymax;
    model = glm::translate(model, glm::vec3(-12.0f, 60.0f, 0.0f));
    model = glm::rotate(model, giroBrazo * toRadians, glm::vec3(-1.0f, 0.0f, 0.0f));
    modelauxBaymax = model;
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    BrazoDerechoBaymax.RenderModel();

    model = modelauxBaymax;
    model = glm::translate(model, glm::vec3(-13.0f, -30.0f, 0.0f));
    model = glm::rotate(model, giraMano * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
    modelauxBaymax = model;
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    ManoBaymax.RenderModel();
}

```

Fig.60. Impresión de modelo de acuerdo a las banderas afuera del tablero.

```

model = modelauxBaymax;
model = glm::translate(model, glm::vec3(-5.0f, -4.0f, 0.0f));
model = glm::rotate(model, sin(giraDedos) * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
DedoMedioBaymax.RenderModel();

model = modelauxBaymax;
model = glm::translate(model, glm::vec3(-5.0f, -4.0f, 2.0f));
model = glm::rotate(model, sin(giraDedos) * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
DedoIzquierdoBaymax.RenderModel();

model = modelauxBaymax;
model = glm::translate(model, glm::vec3(-5.0f, -4.0f, -2.0f));
model = glm::rotate(model, sin(giraDedos) * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
DedoDerechoBaymax.RenderModel();

model = modelauxBaymax;
model = glm::translate(model, glm::vec3(2.0f, -4.0f, 2.0f));
model = glm::rotate(model, sin(giraDedos) * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
DedoGordoBaymax.RenderModel();
}

```

Fig.61. Impresión de modelo de acuerdo a las banderas afuera del tablero.

```

model = glm::mat4(1.0); // Personaje Secundario2 afuera del tablero
model = glm::translate(model, glm::vec3(0.0f, -0.8f, -8.0f));
if (PerPrincipal1 == true) {
    model = glm::mat4(1.0); // Baymax Animado
    model = glm::translate(model, glm::vec3(0.0f, -0.8f, -8.0f));
    model = glm::scale(model, glm::vec3(0.1f, 0.1f, 0.1f));
    modelauxBaymax = model;
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    BaymaxBlanco_M.RenderModel();
}

```

Fig.62. Impresión de modelo de acuerdo a las banderas afuera del tablero.

```

    }
    else if (PerPrincipal2 == true) {
        model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
        modelaux2 = model;
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
        Perry.RenderModel();
    }
    else if (PerPrincipal3 == true) {
        model = glm::scale(model, glm::vec3(0.6f, 0.6f, 0.6f));
        modelaux2 = model;
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
        Bombon.RenderModel();
    }
}

```

Fig.63. Impresión de modelo de acuerdo a las banderas afuera del tablero.

Animación Baymax

Uno de los personajes principales cuenta con una animación mientras está en espera de ser seleccionado para jugar, esta animación es la clásica del saludo entre Baymax y Hiro Hamada en la película de “6 Grandes Heroes”. Para realizar la animación se tiene una bandera booleana nombrada mueveBrazo, la cuál nos indica si el brazo de Baymax debe subir o no. El brazo sube hasta que llega a una posición de 85° en donde se detiene y activa otra bandera de tipo boolean mueveMano que nos indica que la mano debe girar para poder dar el puño, una vez que giró y está en la posición correcta la bandera mueveBrazo vuelve a activarse para que el saludo concluya. Después de realizar todas estas acciones los pasos se repiten para que el personaje esté saludando todo el tiempo hasta que sea su turno de jugar.

```

//Animacion BAYMAX "SALUDO"
if (mueveBrazo) {
    if (giroBrazo < 85 or giroBrazo > 90) {
        giroBrazo += giroBrazoOffset * deltaTime;
    }
    else { mueveBrazo = !mueveBrazo; mueveMano = true; }
    if (giroBrazo > 180) { giroBrazo = 0.0; giraDedos = 0.0; giraMano = 0.0; }
}
else {
    if (mueveMano) {
        if (giraMano < 90) { giraMano += giraManoOffset * deltaTime; }
        else {
            giroBrazo = 91; mueveMano = -!mueveMano;
            giraDedos += 0.5f * deltaTime; mueveBrazo = true;
        }
    }
}
}

```

Fig.64. Líneas para animar a Baymax.

Animación Bombón

Otro personaje principal presente en la escena del tablero es el de bombón el cual tiene una animación propia para la cual primero se definieron las variables a utilizar las de movimiento en los diferentes ejes, para mover al personaje, la rotación para que el brazo tenga la animación de que sube y las variables para que la pierna derecha también vaya hacia arriba en y, para obtener la pose de que vuela, además de sus respectivos offset, el tiempo que se usará para la función en la que

se logra que la trayectoria en la que vuela bombón es en forma de corazón y la deltaT para ajustar la velocidad a la que completa la trayectoria.

```
// VARIABLES PARA ANIMACIÓN DE BOMBON
float movBx, movBy, movBz, rotB;
float movOffset, movOffsetp, movOffsetb;
float rotbombonOffset, rotbrazobomOffset;
float t, deltaT;
float subiry, rotbrazo, bajarbrazo;

Model Bombon2, BrazoI, PiernaD;
```

Fig.65. Variables y modelos para la animación de Bombón.

```
// Modelo de Bombon por partes
Bombon2 = Model();           Bombon2.LoadModel("Models/bombon2.obj");
BrazoI = Model();            BrazoI.LoadModel("Models/brazoI.obj");
PiernaD = Model();           PiernaD.LoadModel("Models/piernaD.obj");
```

Fig.66. Declaración y ruta de donde se encuentran los modelos de Bombón.

```
// Variables de movimeinto para Bombon
movBx = 0.0f; movBy = 0.0f; movBz = 0.0f; rotB = 0.0f; rotbrazo = 0.0f;
movOffset = 0.03f; movOffsetp = 0.0003f; rotbombonOffset = 0.02f;
subiry = 0.0f; rotbrazobomOffset = 0.04f; movOffsetb = 0.0004f; bajarbrazo = 0.0f;
t = 0.0f; deltaT = 0.01f;
```

Fig.67. Inicialización de las variables de Bombón.

Para la animación de bombon primero utilizando rotbrazo fue aumentando el contador hasta que este llegara a posicionarse en un ángulo de -120 grados, posteriormente para que el brazo levantado se acomodará de mejor manera también había que bajarlo unos centímetros por ello se posicionó 0.5 hacia abajo.

Por ultimo va subiendo bombon en el eje y hasta llegar a la altura de 30 donde comenzará a trazar una trayectoria en forma de corazón usando la función $16 * \text{pow}(\sin(t), 3)$ para el movimiento en x y la función $-25.0f + (13 * \cos(t) - 5 * \cos(2 * t) - 2 * \cos(3 * t) - \cos(4 * t))$ para el movimiento en z, aumentando la variable t, cuando t sea igual al valor de $2 * \pi$, la variable t se reinicia.

```
// Animacion de Bombon
if (rotbrazo > -120.0f) { rotbrazo -= rotbrazobomOffset * deltaTime; }
if (bajarbrazo > -0.5f) { bajarbrazo -= movOffsetb * deltaTime; }
if (movBy < 30.0f) { movBy += movOffset * deltaTime;
    if (rotB < 30) { rotB += rotbombonOffset * deltaTime; }
    if (subiry < 0.7f) { subiry += movOffsetp * deltaTime; } }
else { movBx = (16 * pow(sin(t), 3));
    movBz = -25.0f + (13 * cos(t) - 5 * cos(2 * t) - 2 * cos(3 * t) - cos(4 * t));
    t += deltaT;
    if (t > 2 * M_PI) t = 0.0;}
```

Fig.68. Animación de Bombón.

```

if (PerPrincipal1 == true) {
    model = glm::mat4(1.0f);
    model = glm::rotate(model, rotB * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
    model = glm::translate(model, glm::vec3(65.0f + movBx, -0.8f + movBy, movBz));
    model = glm::scale(model, glm::vec3(0.6f, 0.6f, 0.6f));
    modelaux2 = model;
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    Bombon2.RenderModel();

    model = modelaux2;
    model = glm::translate(model, glm::vec3(0.2f, 4.0f + bajarbrazo, -0.8f));
    model = glm::rotate(model, rotbrazo * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    BrazoI.RenderModel();

    model = modelaux2;
    model = glm::translate(model, glm::vec3(-0.3f, 2.3f + subiry, -0.5f));
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    PiernaD.RenderModel();
}

```

Fig.69. Posicionamiento de los modelos de Bombón.

Cámaras:

→ Una cámara en 3era persona ligada al avatar.

Para la cámara en tercera persona lo primero fue realizar la declaración de las variables a utilizar como se ve en la Fig. 70 será un vector de posición, la distancia y la altura a la que se encuentra la cámara en tercera persona. También declaramos los métodos que vamos a utilizar para realizar el cambio.

```

// Variables para cámara en tercera persona
glm::vec3 targetPosition;
float cameraDistance;
float cameraHeight;

```

Fig.70. Variables para la cámara en tercera persona.

```

// Métodos para cambiar entre modos de cámara
void toggleAerialView();
bool isInAerialView() { return isAerialView; }

glm::vec3 getCameraPosition();
glm::vec3 getCameraDirection();
glm::mat4 calculateViewMatrix();

```

Fig.71. Métodos para el cambio entre cámaras.

En la función Camera inicializamos las variables antes declaradas para después actualizar la cámara obteniendo las coordenadas de la posición del personaje y el ángulo en radianes para que la cámara tenga la misma rotación que el personaje, finalmente la posicionamos detrás del personaje para que lo vaya siguiendo.

```

Camera::Camera(glm::vec3 startPosition, glm::vec3 startUp, GLfloat startYaw)
{
    position = startPosition;
    worldUp = startUp;
    yaw = startYaw;
    pitch = startPitch;
    front = glm::vec3(0.0f, 0.0f, -1.0f);

    moveSpeed = startMoveSpeed;
    turnSpeed = startTurnSpeed;

    // Inicializar valores para tercera persona
    cameraDistance = 8.0f;    // Ajusta según necesites
    cameraHeight = 8.0f;     // Ajusta según necesites

    // Inicializar valores para vista aérea
    isAerialView = false;
    aerialHeight = 500.0f;    // Altura de la vista aérea
    aerialMoveSpeed = 0.5f;   // Velocidad de movimiento en vista aérea

    update();
}

```

Fig.72. Función para la cámara en tercera persona.

```

// Implementación del método updateThirdPerson
void Camera::updateThirdPerson(const glm::vec3& characterPosition, float characterRotation) {
    // Actualizar la posición objetivo (personaje)
    targetPosition = characterPosition;

    // Calcular la posición de la cámara basada en la rotación del personaje
    float angleInRadians = glm::radians(characterRotation);

    // Posicionar la cámara detrás del personaje
    position.x = targetPosition.x - (cameraDistance * sin(angleInRadians));
    position.z = targetPosition.z - (cameraDistance * cos(angleInRadians));
    position.y = targetPosition.y + cameraHeight;

    // Hacer que la cámara mire hacia el personaje
    front = glm::normalize(targetPosition - position);

    // Actualizar los vectores right y up
    right = glm::normalize(glm::cross(front, worldUp));
    up = glm::normalize(glm::cross(right, front));
}

```

Fig.73. Función para actualizar la cámara en tercera persona.

Por último si se va a cambiar de la cámara en tercera persona a la cámara aérea se guarda la última posición antes de realizar el cambio. Para llevar a cabo el cambio declaramos las variables de la Fig. 74 inicializandolas en false, luego verificamos que se haya presionado la tecla V y que nos indique si está o no presionada, si esto es verdad se actualiza la cámara, así como la proyección.

```

void Camera::toggleAerialView() {
    isAerialView = !isAerialView;
    if (isAerialView) { // Guardar la posición actual antes de cambiar a vista aérea
        position.y = aerialHeight; // Apuntar directamente hacia abajo
        front = glm::vec3(0.0f, -1.0f, 0.0f);
        right = glm::vec3(1.0f, 0.0f, 0.0f);
        up = glm::vec3(0.0f, 0.0f, -1.0f);
    }
}

```

```
bool useOrthoProjection = false;
bool vKeyPressed = false;
```

```
if (mainWindow.getKeys()[GLFW_KEY_V] && !vKeyPressed) { // Cambiar entre vistas con tecla V
    camera.toggleAerialView();
    useOrthoProjection = camera.isInAerialView();
    vKeyPressed = true;
}
if (!mainWindow.getKeys()[GLFW_KEY_V]) {vKeyPressed = false;}

// Actualizar Cámara
if (camera.isInAerialView()) {camera.updateAerialView(mainWindow.getKeys(), deltaTime);}
else {camera.updateThirdPerson(glm::vec3(movX, 0.0f, movZ), giro);}

// Actualizar proyección
glm::mat4 projection;
if (useOrthoProjection) {
    float aspectRatio = (GLfloat)mainWindow.getBufferWidth() / mainWindow.getBufferHeight();
    float orthoSize = 100.0f;
    projection = glm::ortho(-orthoSize * aspectRatio, orthoSize * aspectRatio, -orthoSize, orthoSize, 0.1f, 1000.0f);}
else {projection = glm::perspective(45.0f, (GLfloat)mainWindow.getBufferWidth() / mainWindow.getBufferHeight(), 0.1f, 1000.0f);}
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection)); //Actualizar el shader con la nueva matriz de proyección
```

Fig.74. Función y variables para alternar la cámara en tercera persona.

→ Una cámara aérea que permitirá ver toda la ciudad desde arriba y se podrá mover solo de forma ortogonal

Con la cámara aérea se realiza un proceso parecido, declaramos las variables que vamos a utilizar de la Fig. 75 la primera para saber si se activa o no la vista aérea, la segunda para altura y la tercera para el movimiento de la cámara de forma ortogonal.

```
// Variables para cámara aérea
bool isAerialView;
float aerialHeight;
float aerialMoveSpeed;
```

Fig.75. Variables para la cámara aérea.

Igualmente en la función Camera se inicializan las variables para la cámara aérea poniendo como false a la variable isAerialView, en 500 la altura y en 0.5 la velocidad con que se moverá de manera ortogonal la vista.

```

Camera::Camera(glm::vec3 startPosition, glm::vec3 startUp, GLfloat startYaw)
{
    position = startPosition;
    worldUp = startUp;
    yaw = startYaw;
    pitch = startPitch;
    front = glm::vec3(0.0f, 0.0f, -1.0f);

    moveSpeed = startMoveSpeed;
    turnSpeed = startTurnSpeed;

    // Inicializar valores para tercera persona
    cameraDistance = 8.0f;    // Ajusta según necesites
    cameraHeight = 8.0f;     // Ajusta según necesites

    // Inicializar valores para vista aérea
    isAerialView = false;
    aerialHeight = 500.0f;    // Altura de la vista aérea
    aerialMoveSpeed = 0.5f;   // Velocidad de movimiento en vista aérea

    update();
}

```

Fig.76. Función para la cámara aérea.

En esta función realizamos el cálculo de la velocidad en qué se moverá la cámara y se guardan las teclas con las que se mueve la cámara de forma ortogonal como W, S, A y D, y se asigna al vector de posición en y de la cámara el valor de la altura.

```

void Camera::updateAerialView(bool* keys, GLfloat deltaTime) {
    if (!isAerialView) return;
    GLfloat velocity = aerialMoveSpeed * deltaTime;

    // Movimiento ortogonal en el plano XZ
    if (keys[GLFW_KEY_W]) { position.z -= velocity; } // Mover hacia adelante en Z
    if (keys[GLFW_KEY_S]) { position.z += velocity; } // Mover hacia atrás en Z
    if (keys[GLFW_KEY_A]) { position.x -= velocity; } // Mover hacia la izquierda en X
    if (keys[GLFW_KEY_D]) { position.x += velocity; } // Mover hacia la derecha en X
    position.y = aerialHeight; // Mantener la altura constante
}

```

Fig.77. Función actualizar la cámara aérea.

Finalmente, cada vez que se presiona la letra V, se cambia de cámara ya sea a la aérea o dinámica dependiendo de la que se encuentre en el momento en qué va a realizarse este cambio. Si la letra es presionada y antes de presionarla se encontraba en la vista de tercera persona, cambiamos a la vista aérea como vemos en la Fig. 78, actualizando la cámara según sea el caso, así como la proyección.

```

if (mainWindow.getKeys()[GLFW_KEY_V] && !vKeyPressed) { // Cambiar entre vistas con tecla V
    camera.toggleAerialView();
    useOrthoProjection = camera.isInAerialView();
    vKeyPressed = true;}
if (!mainWindow.getKeys()[GLFW_KEY_V]) {vKeyPressed = false;}

// Actualizar Cámara
if (camera.isInAerialView()) {camera.updateAerialView(mainWindow.getKeys(), deltaTime);}
else {camera.updateThirdPerson(glm::vec3(movX, 0.0f, movZ), giro);}

// Actualizar proyección
glm::mat4 projection;
if (useOrthoProjection) {
    float aspectRatio = (GLfloat)mainWindow.getBufferWidth() / mainWindow.getBufferHeight();
    float orthoSize = 100.0f;
    projection = glm::ortho(-orthoSize * aspectRatio, orthoSize * aspectRatio, -orthoSize, orthoSize, 0.1f, 1000.0f);}
else {projection = glm::perspective(45.0f, (GLfloat)mainWindow.getBufferWidth() / mainWindow.getBufferHeight(), 0.1f, 1000.0f);}

glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection)); //Actualizar el shader con la nueva matriz de proyección

```

Fig.78. Cambio y actualización de la cámara aérea.

Audio:

→ Audio continuo tipo soundtrack que siempre se reproducirá en loop

Para incluir el audio tipo soundtrack en nuestro proyecto utilizamos la librería de audio de irrKlang para ello configuramos nuestro proyecto añadiendo los archivos de las carpetas include y lib, posteriormente declaramos en la parte de arriba las líneas que se muestran en la Fig. 79 para poder hacer uso de la biblioteca.

```

#include<iostream>
#include <irrKlang.h>
using namespace irrklang;
#pragma comment(lib, "irrKlang.lib")

```

Fig.79. Declaración para la librería de audio.

Agregamos la variable para encender o apagar la música, definiéndola como un booleano en true. Después se crea la función musicaFondo y si la variable de la música está encendida iniciando el motor de sonido con los parámetros predeterminados con la función createIrrKlangDevice(), verificamos si se inició correctamente y llamamos a play2D dando como parámetros la ruta donde esta el audio que queremos reproducir y el segundo parámetro es para reproducir en bucle. Por último solo mandamos llamar a la función de musicaFondo dentro del main.

```

//Musica
bool On = true;

```

```

int musicaFondo() {
    if (On) {
        On = false;
        ISoundEngine* engine = createIrrKlangDevice();
        if (!engine) {return 0;} engine->play2D("audio/doodle.mp3", true);
        return 0;
    }
}

```

Fig.80. Función para la música de fondo.

```

int main() {
    mainWindow = Window(1366, 768); // 1280, 1024 or 1024, 768
    mainWindow.Initialise();

    CreateObjects();    Casillas_Esquina();    Casillas_Inter();
    Dado_4();           Dado_8();              Centro();
    CreateShaders();    musicaFondo();
}

```

Fig.81. Llamada a la función en el main.

Conclusiones:

Barragán Pilar Diana

El desarrollo de este proyecto me permitió entender la complejidad y dedicación que implica el diseño de entornos interactivos en 3D. La planificación inicial, basada en nuestro documento guía, fue crucial para definir los elementos principales y las funcionalidades. Me sorprendió lo mucho que influyó la gestión de texturas y luces en la experiencia visual final, destacando el uso de pointlights y spotlights para dar realismo a las escenas. En este trabajo también se notó la importancia de la colaboración y cómo el ajuste continuo de nuestras ideas, así como la correcta planeación fue esencial para superar obstáculos técnicos respectivos al proyecto.

Medrano Miranda Daniel Ulises

Trabajar en este proyecto fue una experiencia reveladora que reforzó mis habilidades en programación y gestión de gráficos 3D. Implementar funciones como el lanzamiento de dados y el movimiento del avatar en el tablero me mostró lo fundamental que es la precisión en la simulación física. También, el uso de la biblioteca irrKlang para la música aportó una capa de inmersión muy satisfactoria al ambiente. El proceso de perfeccionar cada detalle, desde los bocetos en Geogebra hasta la integración de modelos animados, me hizo apreciar el valor del refinamiento y la optimización continua en el desarrollo técnico.

Uriarte Ortiz Enrique Yahir

La realización de este proyecto fue un gran desafío que combinó creatividad y conocimientos técnicos. La organización detallada de los modelos 3D y la implementación de luces y texturas, a partir de nuestra documentación original, requirió una planificación meticulosa. Me llamó la atención cómo cada elemento, desde los cambios de textura diurnos y nocturnos hasta la animación de personajes como Baymax, contribuyó a un entorno más dinámico y atractivo. Además, aprendí que el trabajo en equipo y la capacidad de adaptarse a cambios en los requerimientos son esenciales para el éxito de cualquier proyecto ambicioso.